

Distributed Training: TP/SP/EP/CP

Vlad Savinov, YandexGPT

- Model:
 - Parameters ($x_4 + x_2$)
 - Optimizer states (x_8)
 - Gradients (x_4)
 - Activations (checkpoint activations: $2 \times L \times S \times H$ bytes)

- Model:
 - Parameters ($x4 + x2$)
 - Optimizer states ($x8$)
 - Gradients ($x4$)
 - Activations (checkpoint activations: $2 \times L \times S \times H$ bytes)
 - $32b > 32e9 * 18 = 576 \text{ GB}$
 - $235b > 235e9 * 18 = 4.2 \text{ TB}$

- Model:
 - Parameters ($x4 + x2$)
 - Optimizer states ($x8$)
 - Gradients ($x4$)
 - Activations (checkpoint activations: $2xLxSxH$ bytes)
 - $32b > 32e9 * 18 = 576$ GB
 - $235b > 235e9 * 18 = 4.2$ TB
- GPU:
 - 80-190 GB

- Model:
 - Parameters (x4 + x2)
 - Optimizer states (x8)
 - Gradients (x4)
 - Activations (checkpoint activations: $2 \times L \times S \times H$ bytes)
 - $32b > 32e9 * 18 = 576 \text{ GB}$
 - $235b > 235e9 * 18 = 4.2 \text{ TB}$
- GPU:
 - 80-190 GB

⇒ Модель не помещается в GPU, надо разрезать веса/активации

FSDP - влезут активации?

- World size = 512
- 235b модель:
 - YaFSDP: 30 GB под буферы на параметры/градиенты

FSDP - влезут активации?

- World size = 512
- 235b модель:
 - YaFSDP: 30 GB под буферы на параметры/градиенты
 - YaFSDP: $235e9 * 18 / 512 = 8$ GB

FSDP - влезут активации?

- World size = 512
- 235b модель:
 - YaFSDP: 30 GB под буферы на параметры/градиенты
 - YaFSDP: $235e9 * 18 / 512 = 8$ GB
 - На H100 карте остается 40 GB

FSDP - влезут активации?

- World size = 512
- 235b модель:
 - YaFSDP: 30 GB под буферы на параметры/градиенты
 - YaFSDP: $235e9 * 18 / 512 = 8$ GB
 - На H100 карте остается 40 GB
 - Активации для 4096 контекста:
 - Чекпойнтинг: $2 * L * H * S$ bytes = 6 GB – OK
 - Без чекпойнтинга не влезем (надо раз в 10 больше)

FSDP - влезут активации?

- World size = 512
- 235b модель:
 - YaFSDP: 30 GB под буферы на параметры/градиенты
 - YaFSDP: $235e9 * 18 / 512 = 8 \text{ GB}$
 - На H100 карте остается 40 GB
 - Активации для 4096 контекста:
 - Чекпойнтинг: $2 * L * H * S \text{ bytes} = 6 \text{ GB} - \text{OK}$
 - Без чекпойнтинга не влезем (надо раз в 20 больше)
 - Активации для 65536 контекста:
 - Чекпойнтинг: $2 * 94 * 8192 * 65536 / 1e9 = 100 \text{ GB} :($

FSDP - влезут активации?

- World size = 512
- 235b модель:
 - YaFSDP: 30 GB под буферы на параметры/градиенты
 - YaFSDP: $235e9 * 18 / 512 = 8 \text{ GB}$
 - На H100 карте остается 40 GB
 - Активации для 4096 контекста:
 - Чекпойнтинг: $2 * L * H * S \text{ bytes} = 6 \text{ GB} - \text{OK}$
 - Без чекпойнтинга не влезем (надо раз в 20 больше)
 - Активации для 65536 контекста:
 - Чекпойнтинг: $2 * 94 * 8192 * 65536 / 1e9 = 100 \text{ GB} :($

⇒ Для обучения с длинным контекстом FSDP недостаточно, активации занимают много

FSDP - как быть compute-bound

MLP:

$$T_{\text{bwd_comms}} = (H * I_{\text{матрица весов}}) * (2 \text{ bytes}) * (3 \text{ матрицы}) * (2 AG + RS) / (\text{Netw speed})$$

$$T_{\text{bwd_compute}} = (3 * 2_{\text{матмугла в bwd}}) * (2 * H * I * S / N_{\text{gpus флопсов}}) / (\text{FLOPs per gpu})$$

FSDP - как быть compute-bound

MLP:

$$T_{\text{bwd_comms}} = (H * I_{\text{матрица весов}}) * (2 \text{ bytes}) * (3 \text{ матрицы}) * (2 AG + RS) / (\text{Netw speed})$$

$$T_{\text{bwd_compute}} = (3 * 2_{\text{матм. в bwd}}) * (2 * H * I * S / N_{\text{gpus флопсов}}) / (\text{FLOPs per gpu})$$

$$T_{\text{compute}} > T_{\text{comms}} \Rightarrow S / N_{\text{gpus}} > (\text{FLOPs per gpu}) / (\text{Netw speed})$$

FSDP - как быть compute-bound

MLP:

$$T_{\text{bwd_comms}} = (H * I_{\text{матрица весов}}) * (2 \text{ bytes}) * (3 \text{ матрицы}) * (2 AG + RS) / (\text{Net speed})$$

$$T_{\text{bwd_compute}} = (3 * 2_{\text{матм. в bwd}}) * (2 * H * I * S / N_{\text{gpus флопсов}}) / (\text{FLOPs per gpu})$$

$$T_{\text{compute}} > T_{\text{comms}} \Rightarrow S / N_{\text{gpus}} > (\text{FLOPs per gpu}) / (\text{Netw speed})$$

$$\text{FLOPs} = 800e12, \text{Netw speed} = 400e9 \Rightarrow \text{на каждой карте} > 2e3 \text{ токенов}$$

⇒ Чтобы быть compute-bound, нужно много токенов на карту

[GROK](#)[API](#)[COMPANY](#)[COLOSSUS](#)[CAREERS](#)[NEWS](#)[TRY GROK](#)

[OUR GIGAFACTORY OF COMPUTE]

COLOSSUS



We're running the world's biggest supercomputer, Colossus. Built in 122 days—outpacing every estimate—it was the most powerful AI training system yet. Then we doubled it in 92 days to 200k GPUs. This is just the beginning.

FSDP - как быть compute-bound

Пример:

- World size = 131072
- SeqLen = 2048
- Global Batch Size = $2048 * 131071 = 268\text{M} > \text{Critical Batch Size}^*$

(*) Размер батча определяет signal to noise ratio. С некоторой точки растить батч сайз вредно для качества, так как мы сделаем слишком мало шагов и получим мало сигнала, а оценка градиента и так уже достаточно честная.

FSDP

- 1) GPU poor: не помещаются активации
- 2) GPU rich: нельзя скейлить DP бесконечно, испортим качество или будем communication-bound (еще NCCL может плохо скейлиться на большой DP world size, особенно между кластерами)



Tensor Parallel - пилим активации

Tensor Parallel - пилим активации

а еще веса, состояния
оптимизатора и градиенты

X

0	1
2	3
4	5
6	7

(4, 2)

@

W

10	30
20	40

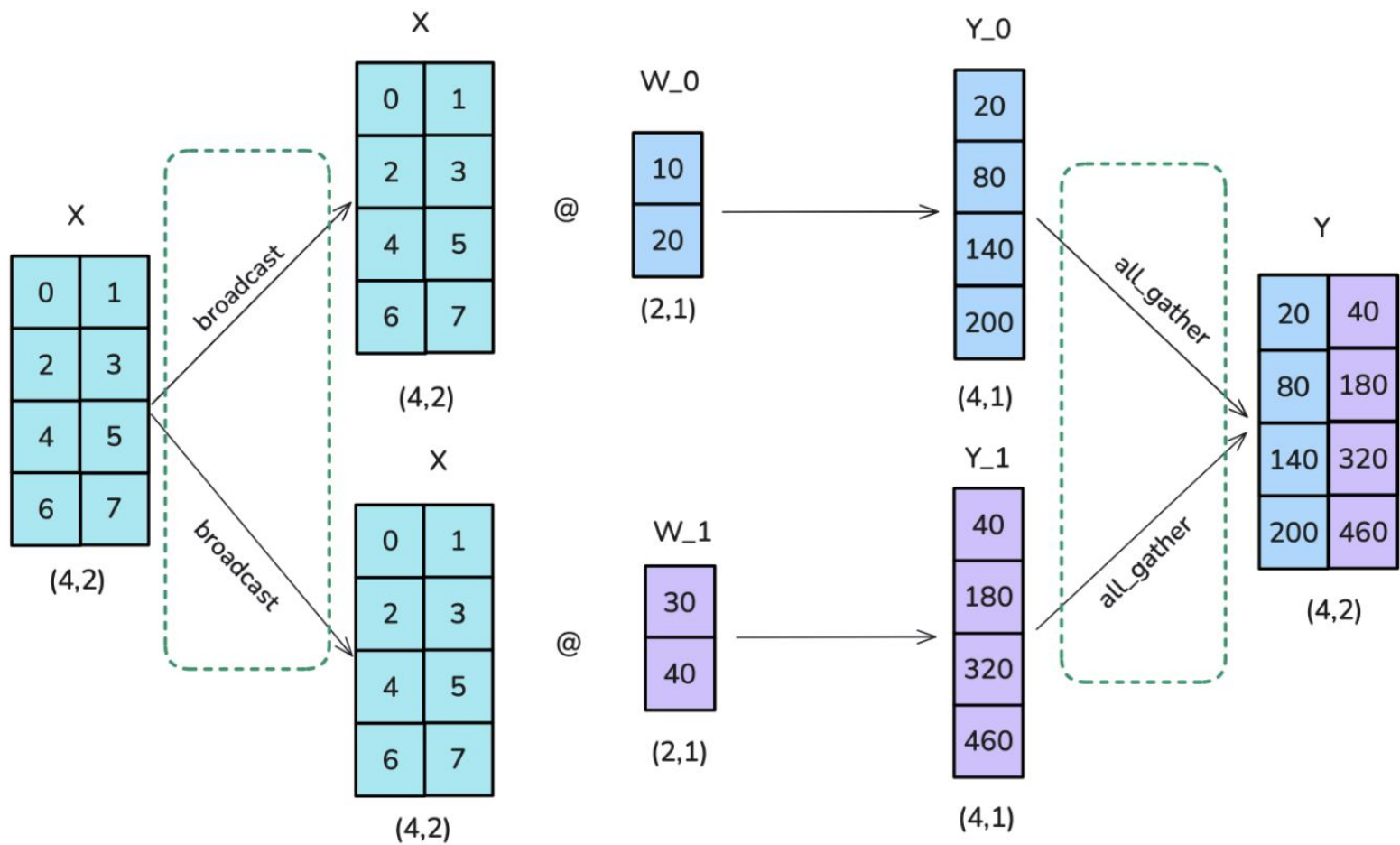
(2, 2)



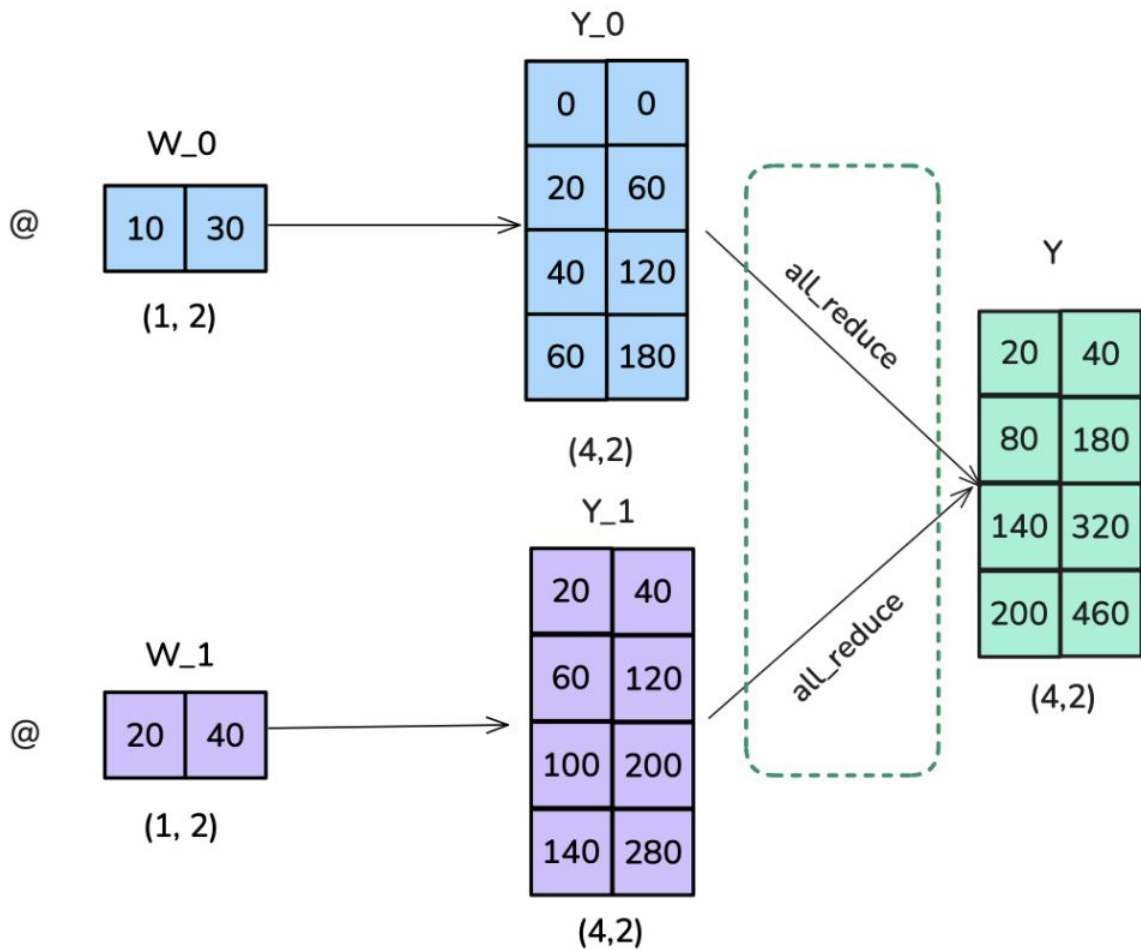
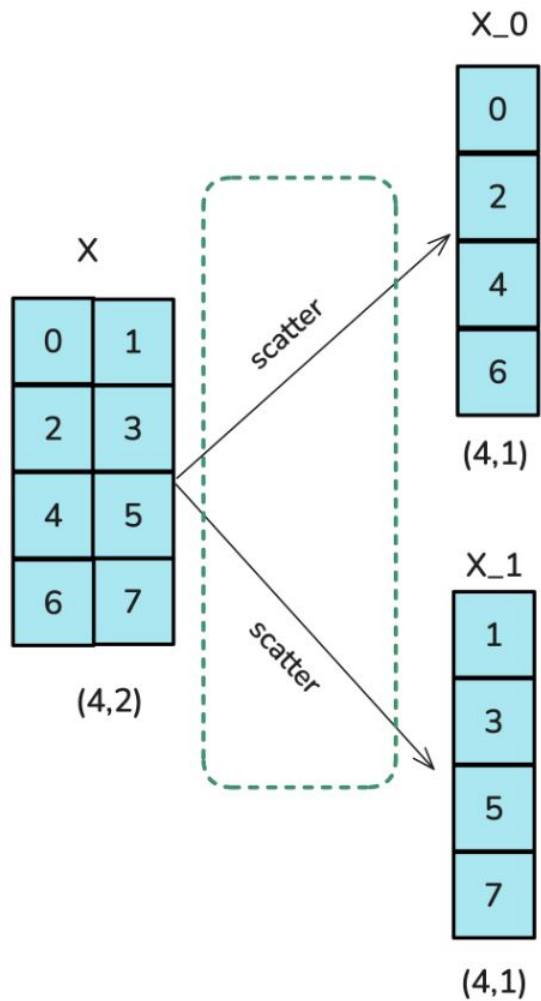
Y

20	40
80	180
140	320
200	460

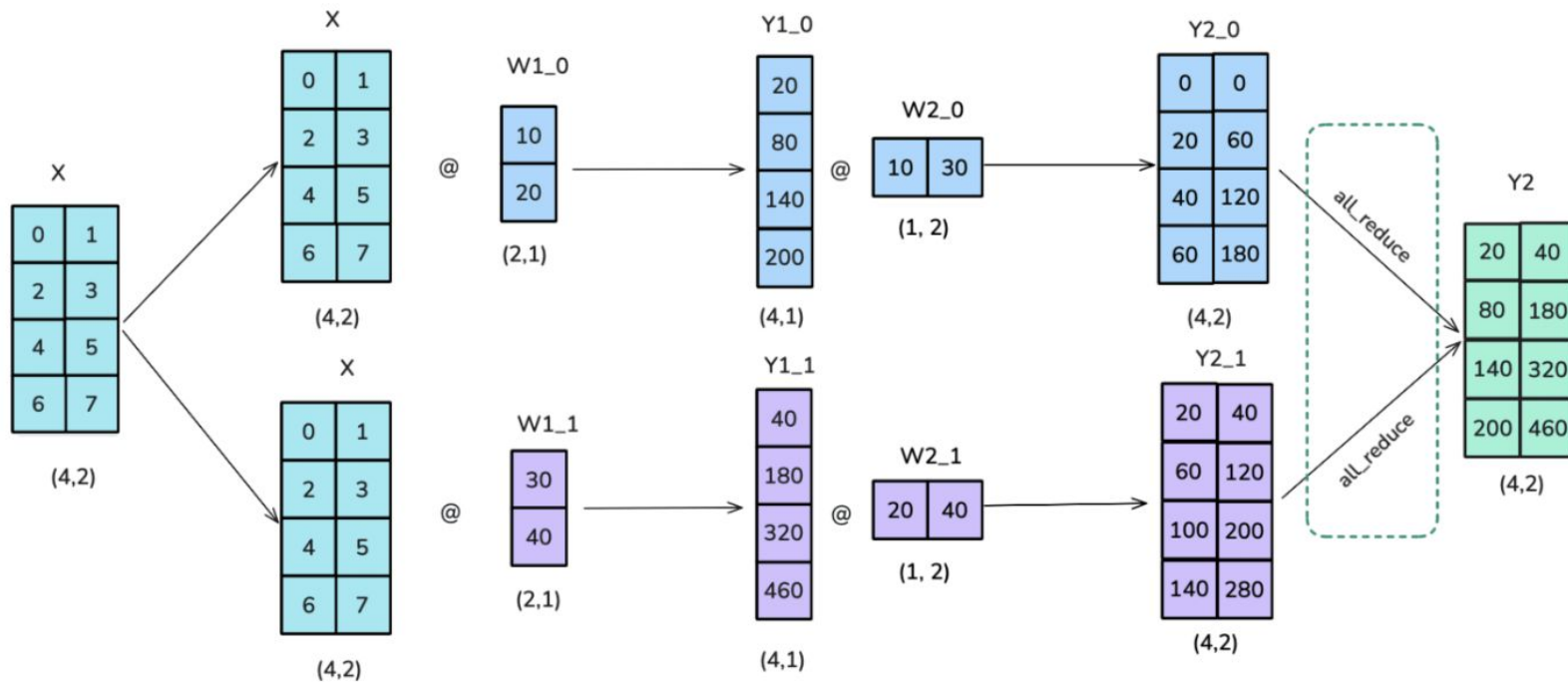
(4, 2)

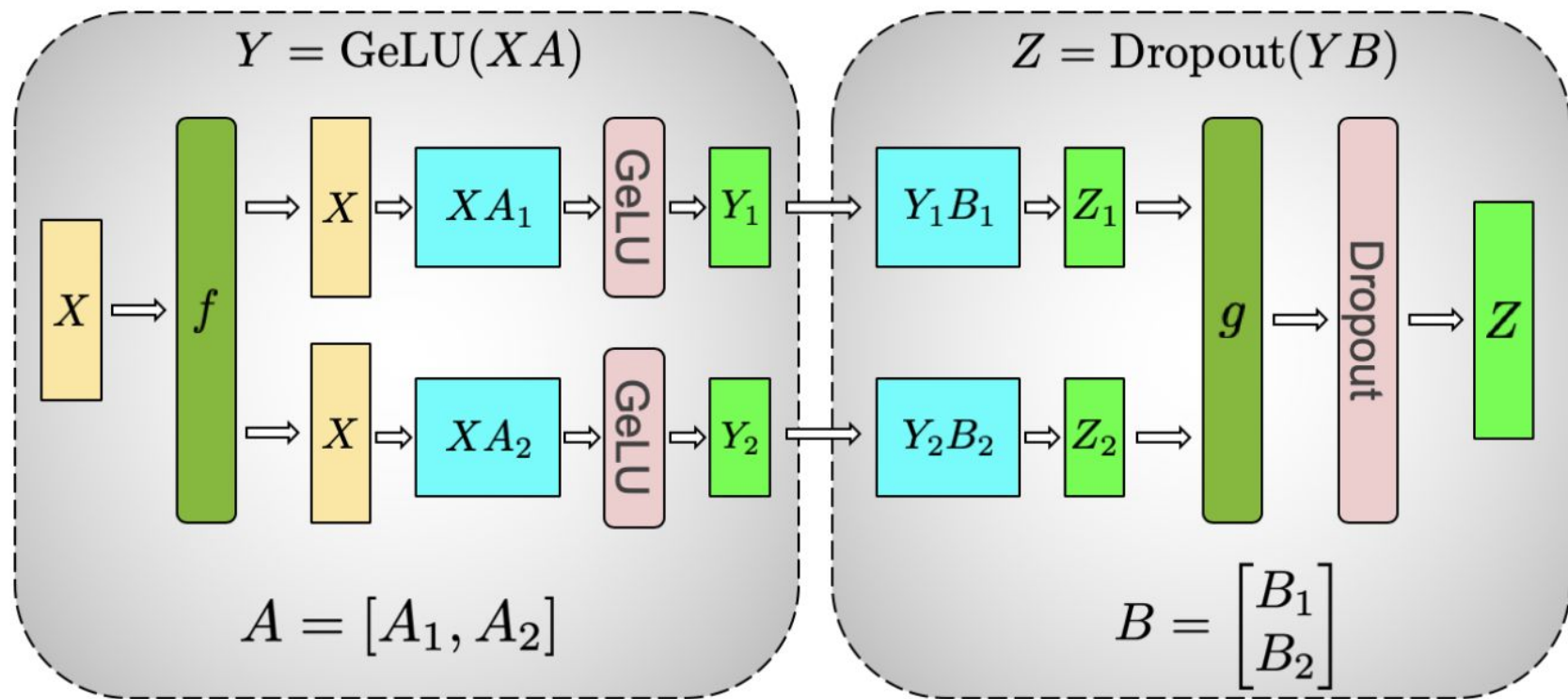


Column linear

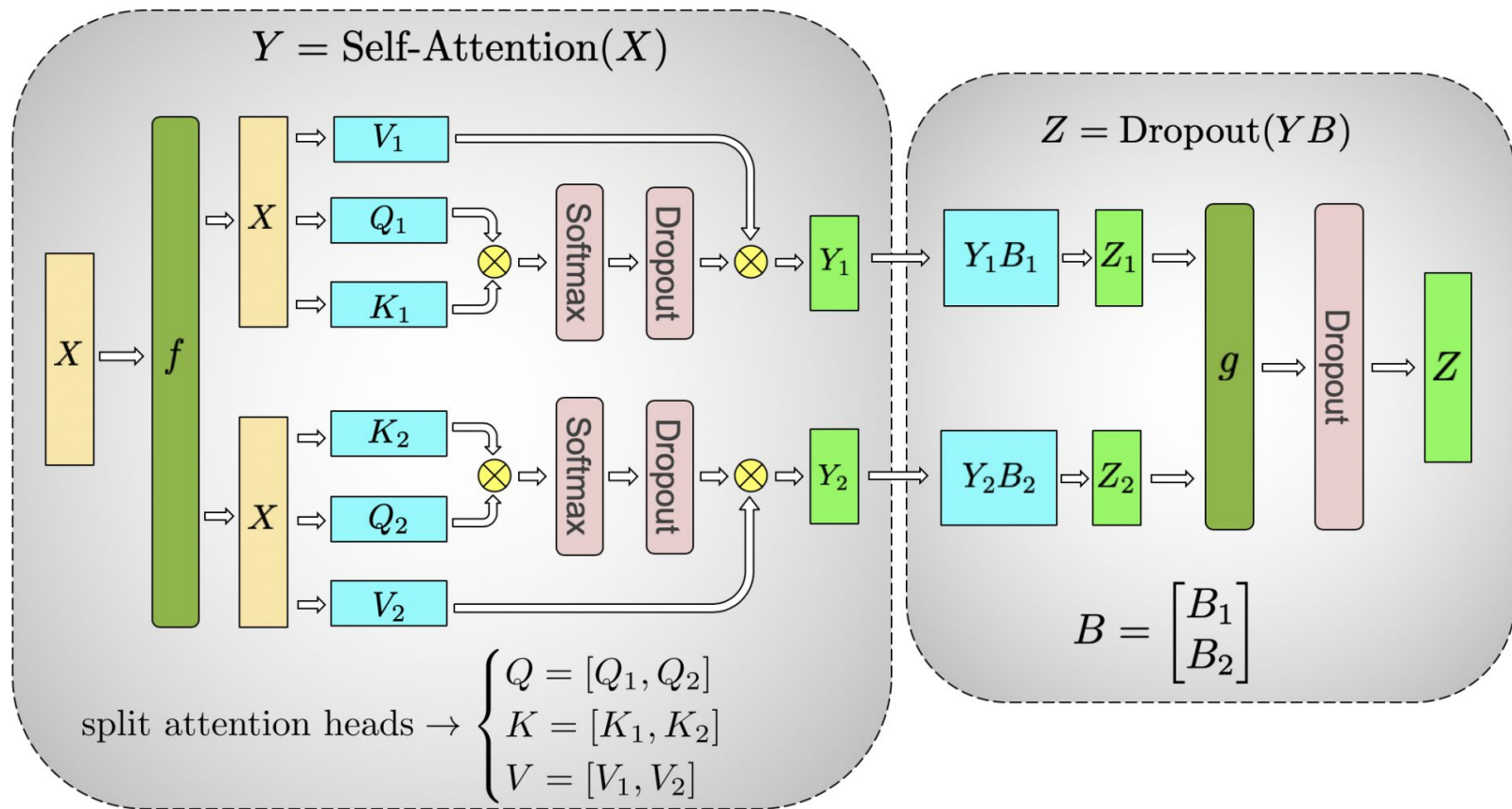


Column-parallel $\rightarrow$ row-parallel





(a) MLP



(b) Self-Attention

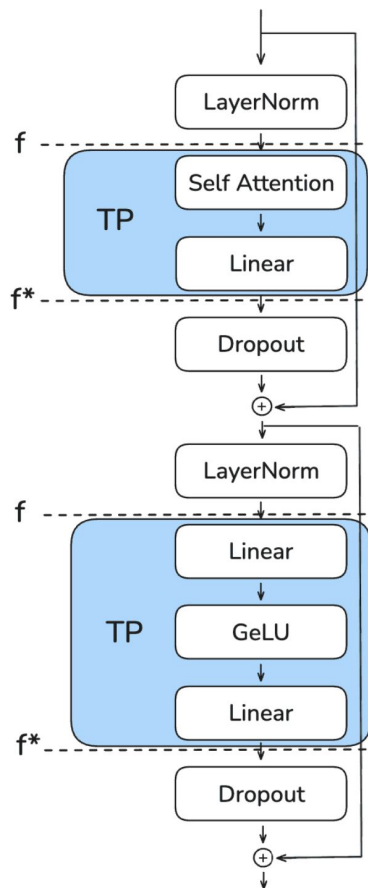
Какой максимальный TP degree можно взять?



Qwen 235b

```
{  
  "head_dim": 128,  
  "hidden_size": 4096,  
  "intermediate_size": 12288,  
  "num_attention_heads": 64,  
  "num_key_value_heads": 4,  
}
```

Tensor Parallel



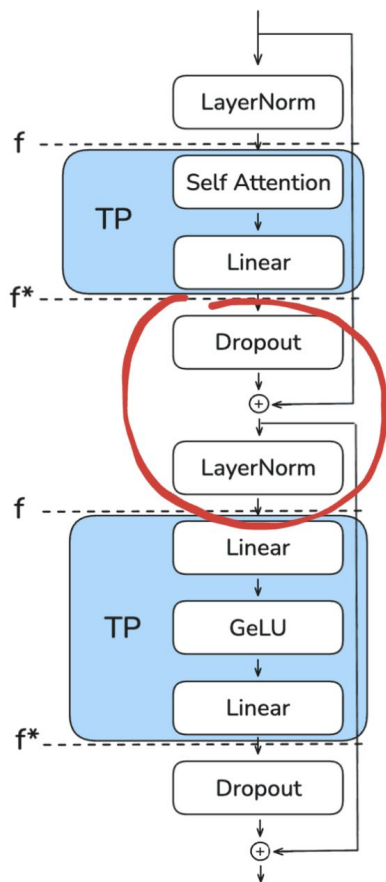
Forward:

- f : no-op, ничего не делаем
- f^* : all-reduce

Backward:

- f^* : no-op, ничего не делаем
- f : all-reduce

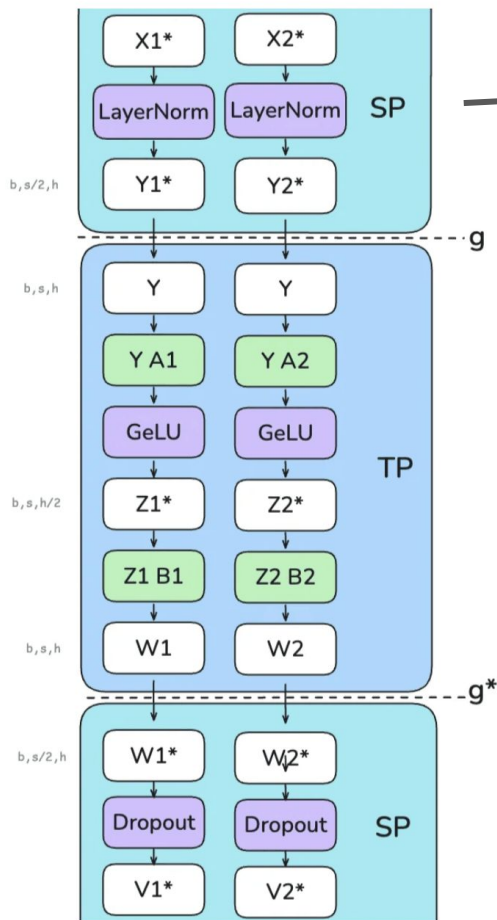
Tensor Parallel



$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

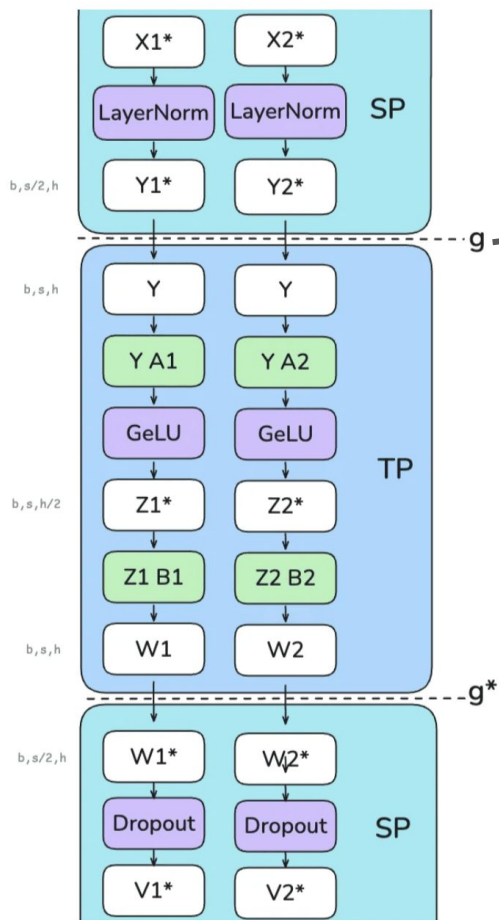
where $\mu = \text{mean}(x)$ and $\sigma^2 = \text{var}(x)$ are computed across hidden dimension h .

Sequence Parallel - лучший друг TP



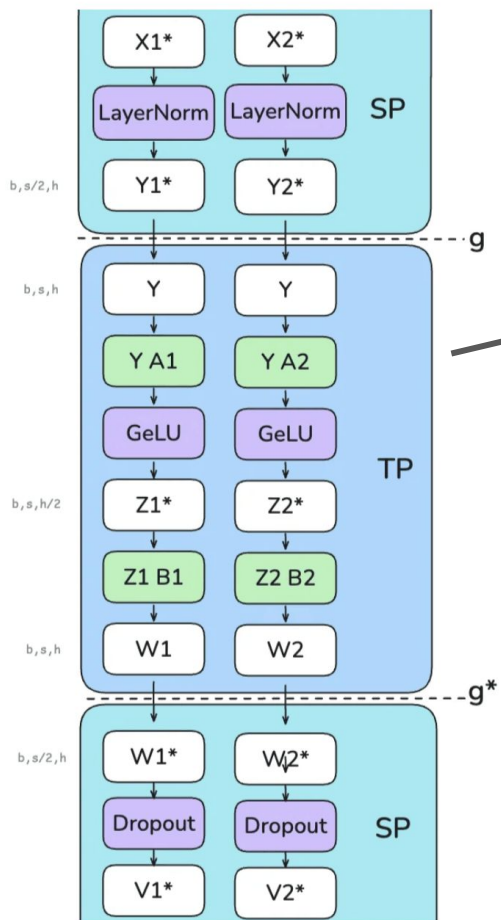
LayerNorm:

- $X1^*$ и $X2^*$ размерности $(b, s/2, h)$
- Каждая GPUшка считает свой кусочек нормы



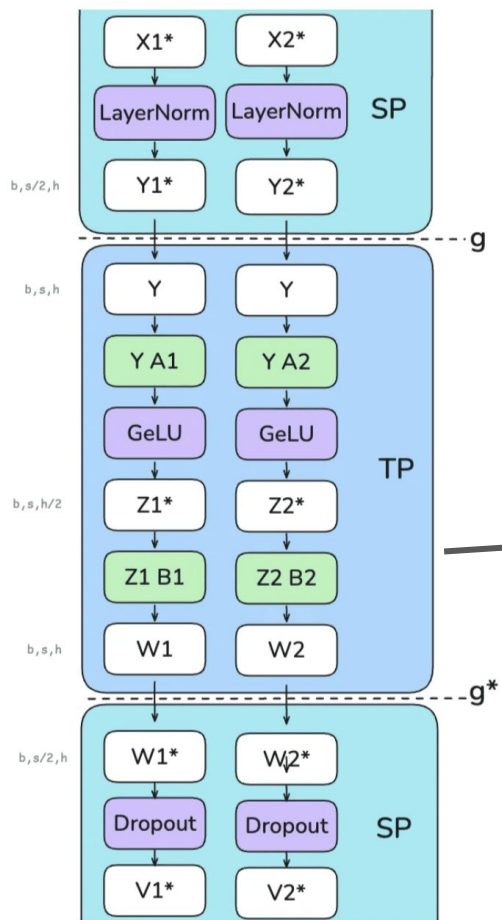
SP $\rightarrow$ TP:

- g = AllGather, собирает $Y1^*$ и $Y2^*$ в Y
- Y с шейпом (b, s, h) - ColumnLinear нужен целый хидден



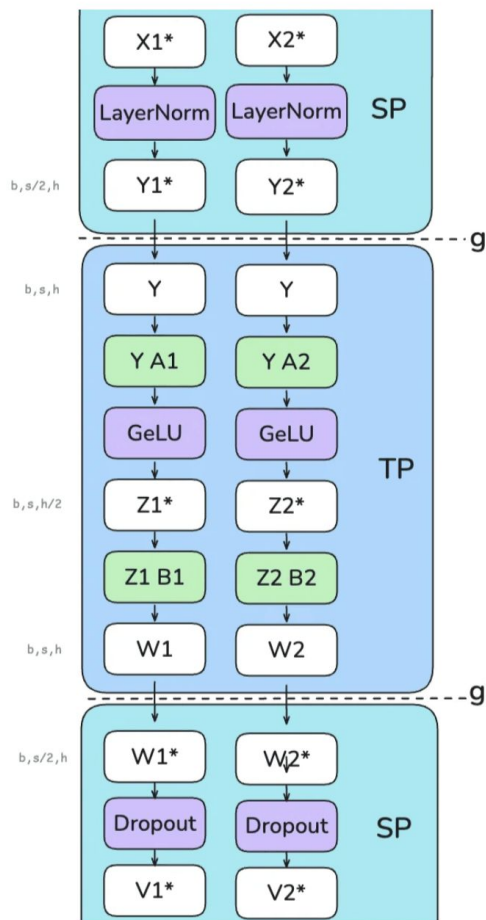
TP:

- A1 и A2 - ColumnLinear
- На выходе получаем $(b, s, h/2)$
- GeLU применяется независимо



TP:

- B1 и B2 - RowLinear
- Превращаем $(b, s, h/2)$ в (b, s, h)



TP → SP

- g^* - ReduceScatter, усредняем результаты для корректности
- На выходе режим по секлену, получаем $(b, s/2, h)$

Region	TP only	TP with SP
Enter TP (column-linear)	<i>h</i> : sharded (weight_out is sharded) <i>s</i> : full	<i>h</i> : sharded (weight_out is sharded) <i>s</i> : all-gather to full
TP region	<i>h</i> : sharded <i>s</i> : full	<i>h</i> : sharded <i>s</i> : full
Exit TP (row-linear)	<i>h</i> : full (weight_out is full + all-reduce for correctness) <i>s</i> : full	<i>h</i> : full (weight_out is full + reduce-scatter for correctness) <i>s</i> : reduce-scatter to sharded
SP region	<i>h</i> : full <i>s</i> : full	<i>h</i> : full <i>s</i> : sharded

Сократили активации в TP раз

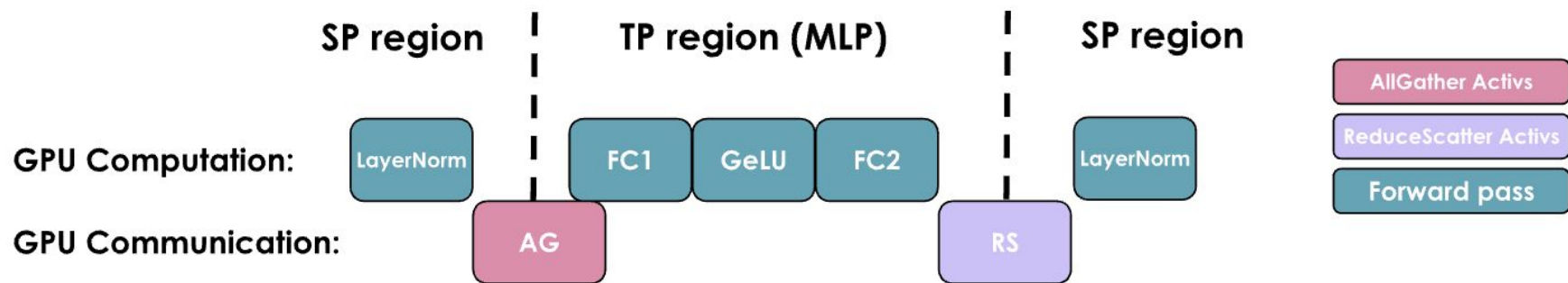
Region	TP only	TP with SP
Enter TP (column-linear)	h : sharded (weight_out is sharded) s : full	h : sharded (weight_out is sharded) s : all-gather to full
TP region	h : sharded s : full	h : sharded s : full
Exit TP (row-linear)	h : full (weight_out is full + all-reduce for correctness) s : full	h : full (weight_out is full + reduce-scatter for correctness) s : reduce-scatter to sharded
SP region	h : full s : full	h : full s : sharded

А что по коммуникациям?

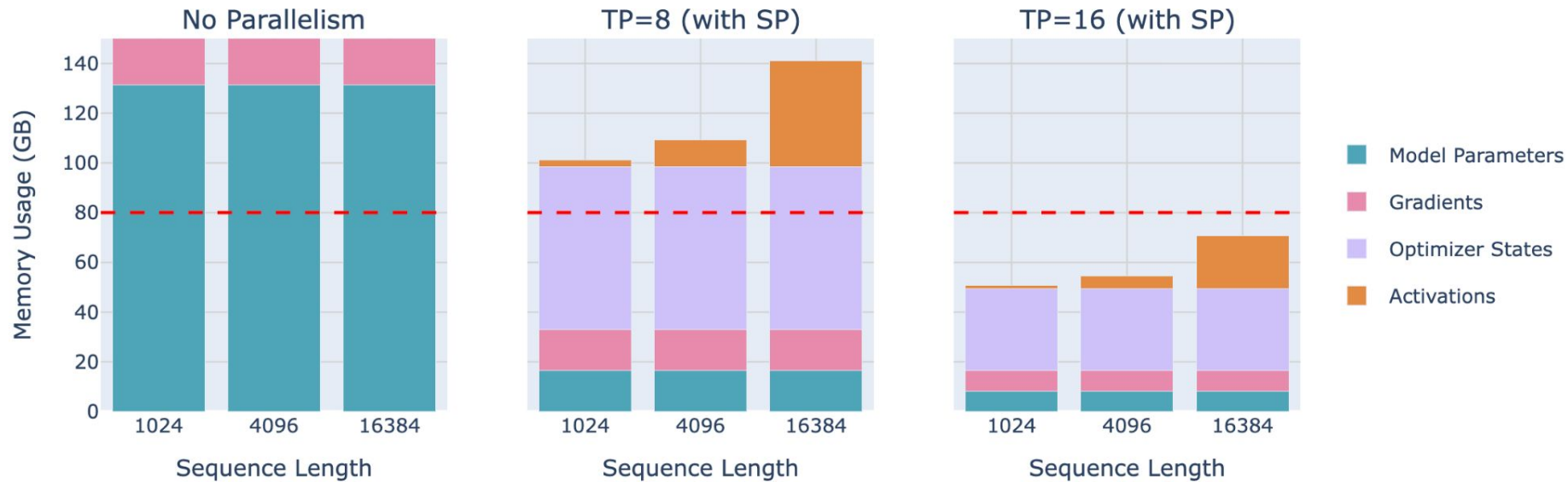
Region	TP only	TP with SP
Enter TP (column-linear)	h : sharded (weight_out is sharded) s : full	h : sharded (weight_out is sharded) s : all-gather to full
TP region	h : sharded s : full	h : sharded s : full
Exit TP (row-linear)	h : full (weight_out is full + all-reduce for correctness) s : full	h : full (weight_out is full + reduce-scatter for correctness) s : reduce-scatter to sharded
SP region	h : full s : full	h : full s : sharded

А что по коммуникациям?

- Было AllReduce + AllReduce
- Стало 2 AllGather + 2 Reduce Scatter



Memory Usage for 70B Model



TPSP - как быть compute-bound

MLP (2 матрицы):

$$T_{\text{fwd_comms}} = (S * H_{\text{активация}}) * (2 \text{ bytes}) * (2 AG + RS) / (\text{Network speed})$$

$$T_{\text{fwd_compute}} = (2 \text{ матмула в fwd}) * (2 * H * I * S / N_{\text{tp_gpus}} \text{ флопсов}) / (\text{FLOPs per gpu})$$

TPSP - как быть compute-bound

$$T_{\text{compute}} > T_{\text{comms}}$$

$$\Rightarrow I > N_{\text{tp_gpus}} * (\text{FLOPs}_{\text{per gpu}}) / (\text{Network speed})$$

FLOPs = 800e12, а сеть бывает разная:

- IB speed = 50e9 (TP > 8) $\Rightarrow I > \text{TP} * 16e3$. Нет таких моделей
- NVLink speed = 400e9 (TP <= 8) $\Rightarrow I > \text{TP} * 2000$. Классика

TPSP - как быть compute-bound

$$T_{\text{compute}} > T_{\text{comms}}$$

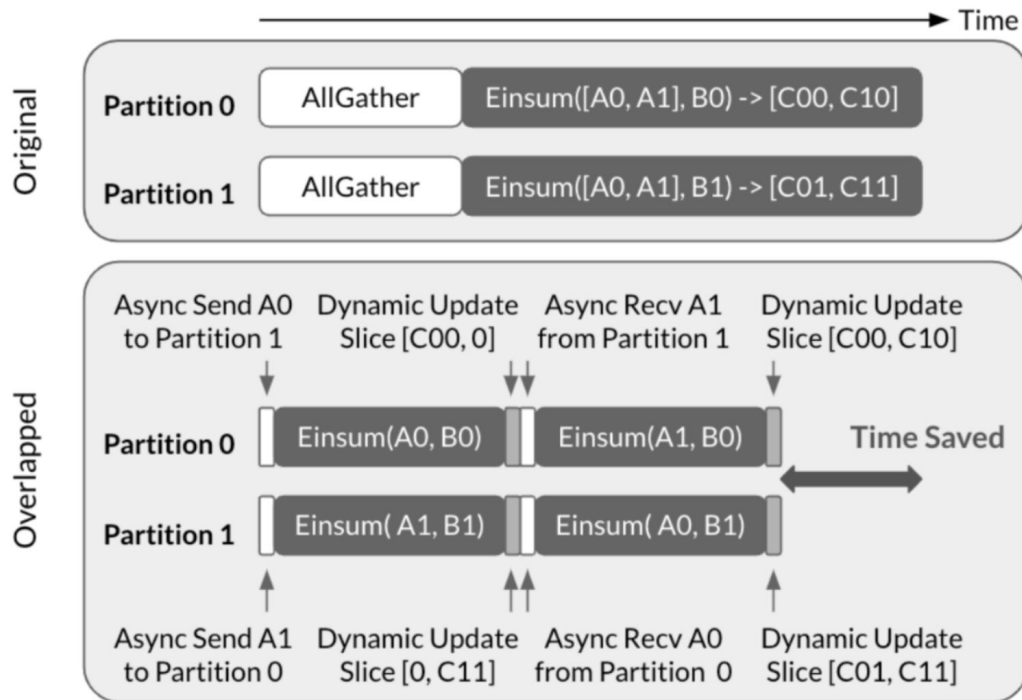
$$\Rightarrow I > N_{\text{tp_gpus}} * (\text{FLOPs}_{\text{per gpu}}) / (\text{Network speed})$$

FLOPs = 800e12, а сеть бывает разная:

- IB speed = 50e9 (TP > 8) $\Rightarrow I > TP * 16e3$. Нет таких моделей
- NVLink speed = 400e9 (TP ≤ 8) $\Rightarrow I > TP * 2000$. Классика

$\Rightarrow$ TP внутри одной ноды

Async TP - задача со звездочкой

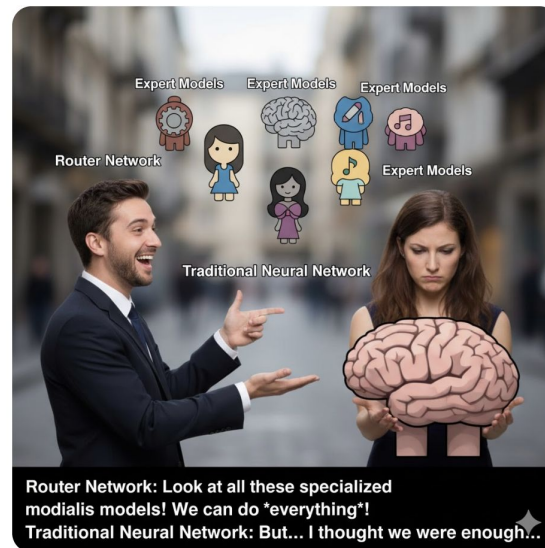


Mixture of Experts

generate a meme about Mixture of Experts (MoE)



Here's a meme about Mixture of Experts (MoE)!



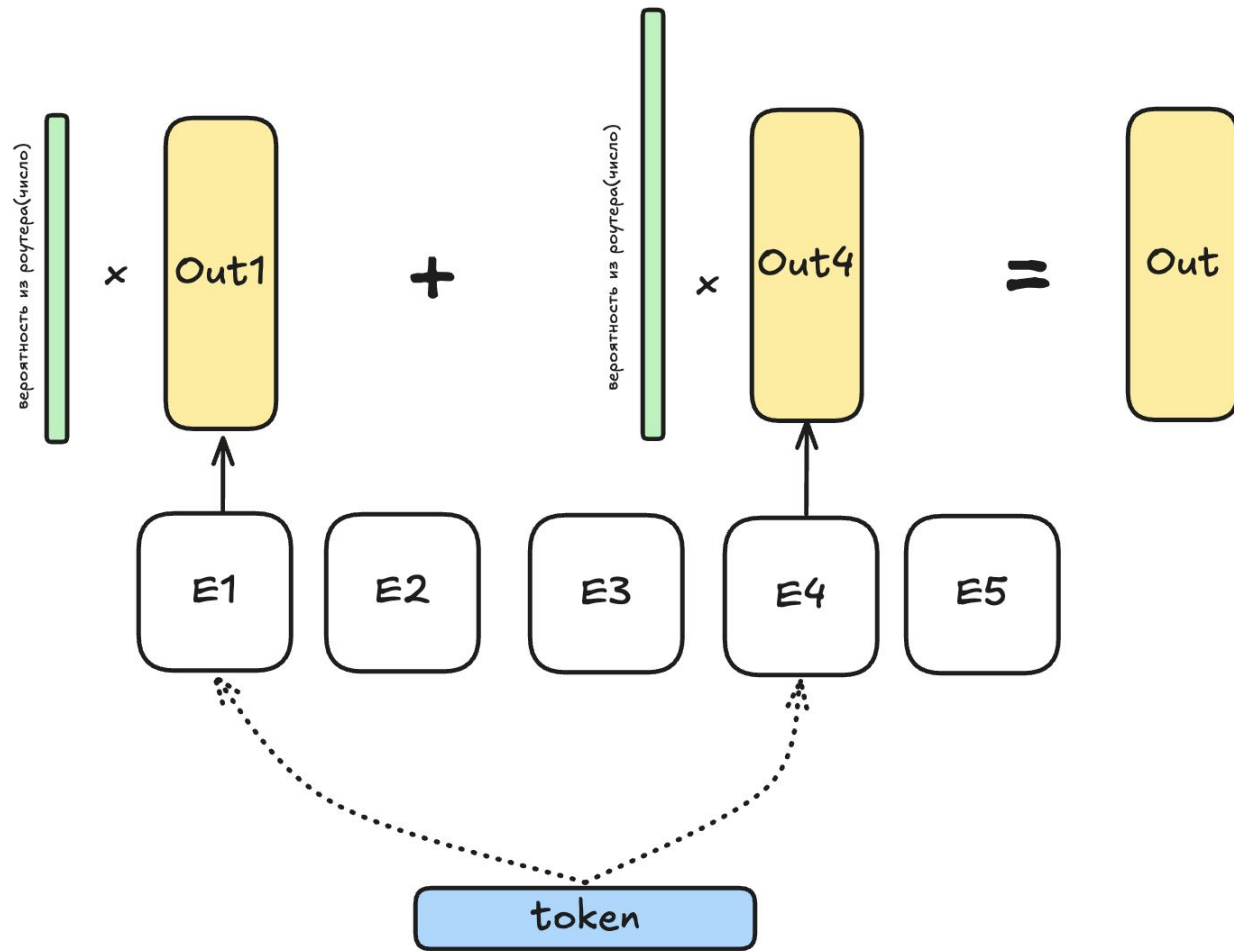
dense transformer block

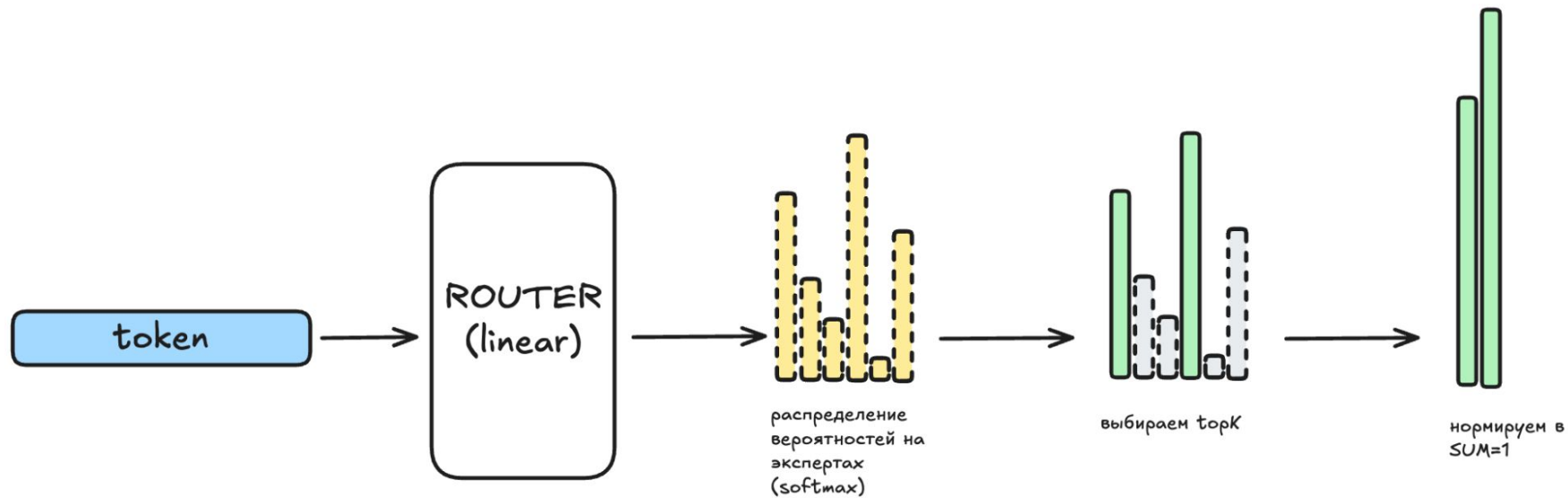
```
1 def dense_block(x):  
2     x = x + attn(attn_norm(x))  
3     x = x + mlp(mlp_norm(x))  
4     return x
```

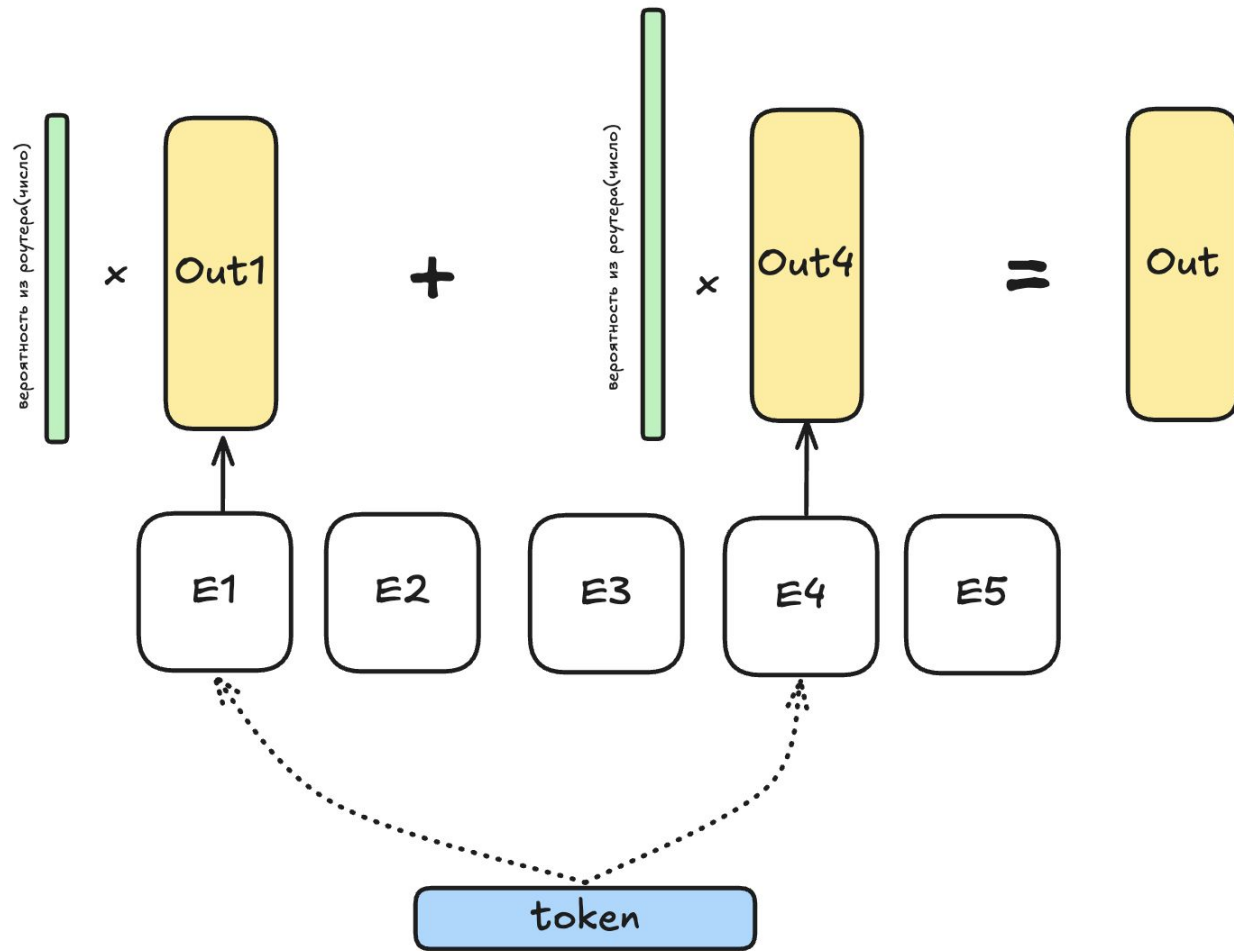
Mixture of Experts = меньше “активных”
параметров, столько же знаний

swiglu

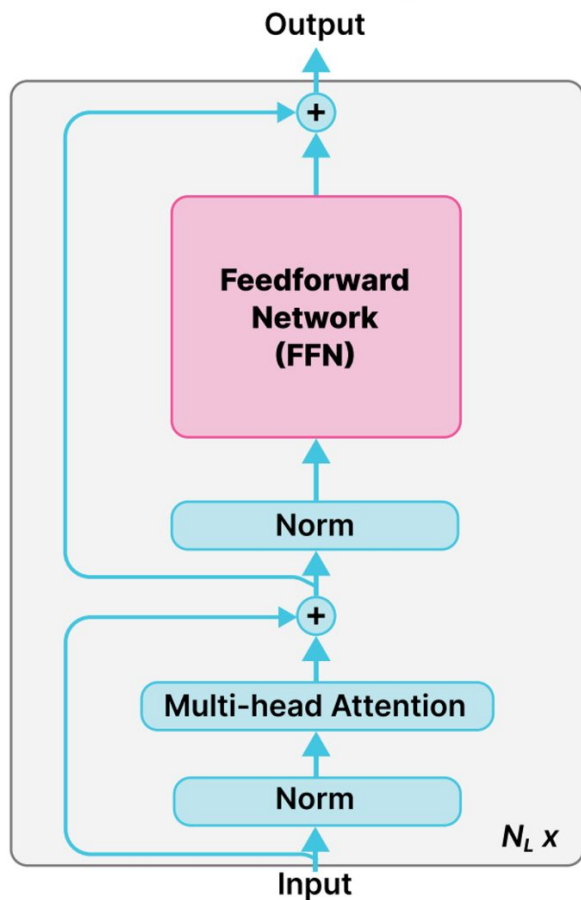
```
1 def swiglu(x, w1, w2, w3):
2     """MLP block.
3     x: (seqlen, hidden)
4     w1: (hidden, intermediate)
5     w2: (hidden, intermediate)
6     w3: (intermediate, hidden)"""
7
8     up = w1(x)
9     gate = silu(w2(x)) # silu(y) = y * sigmoid(y)
10    return w3(up * gate)
11
```



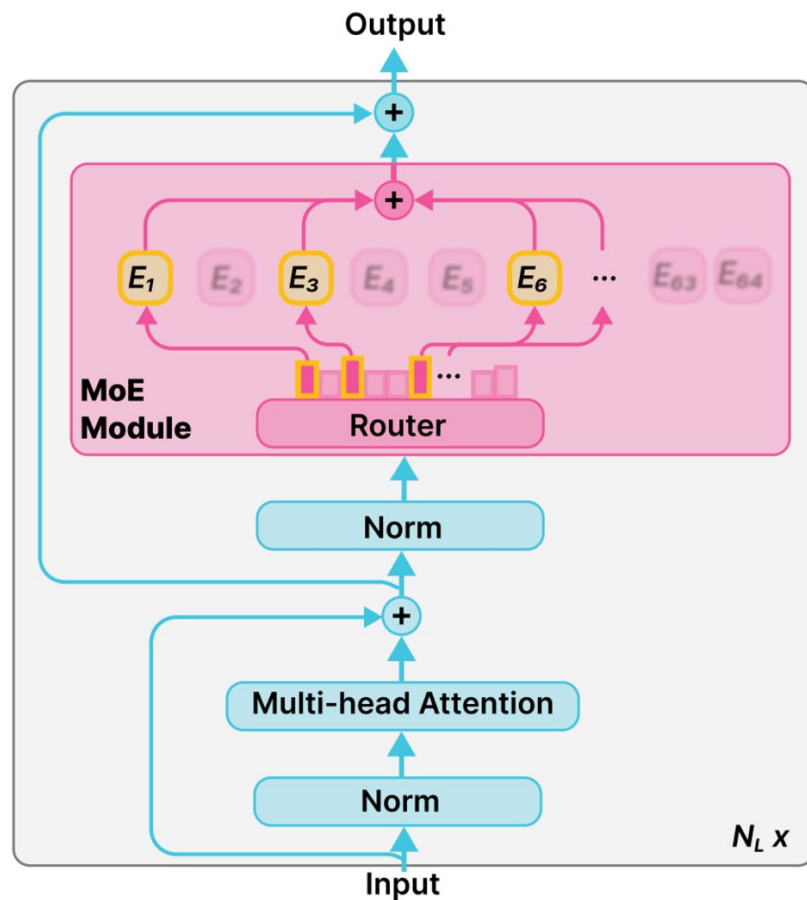


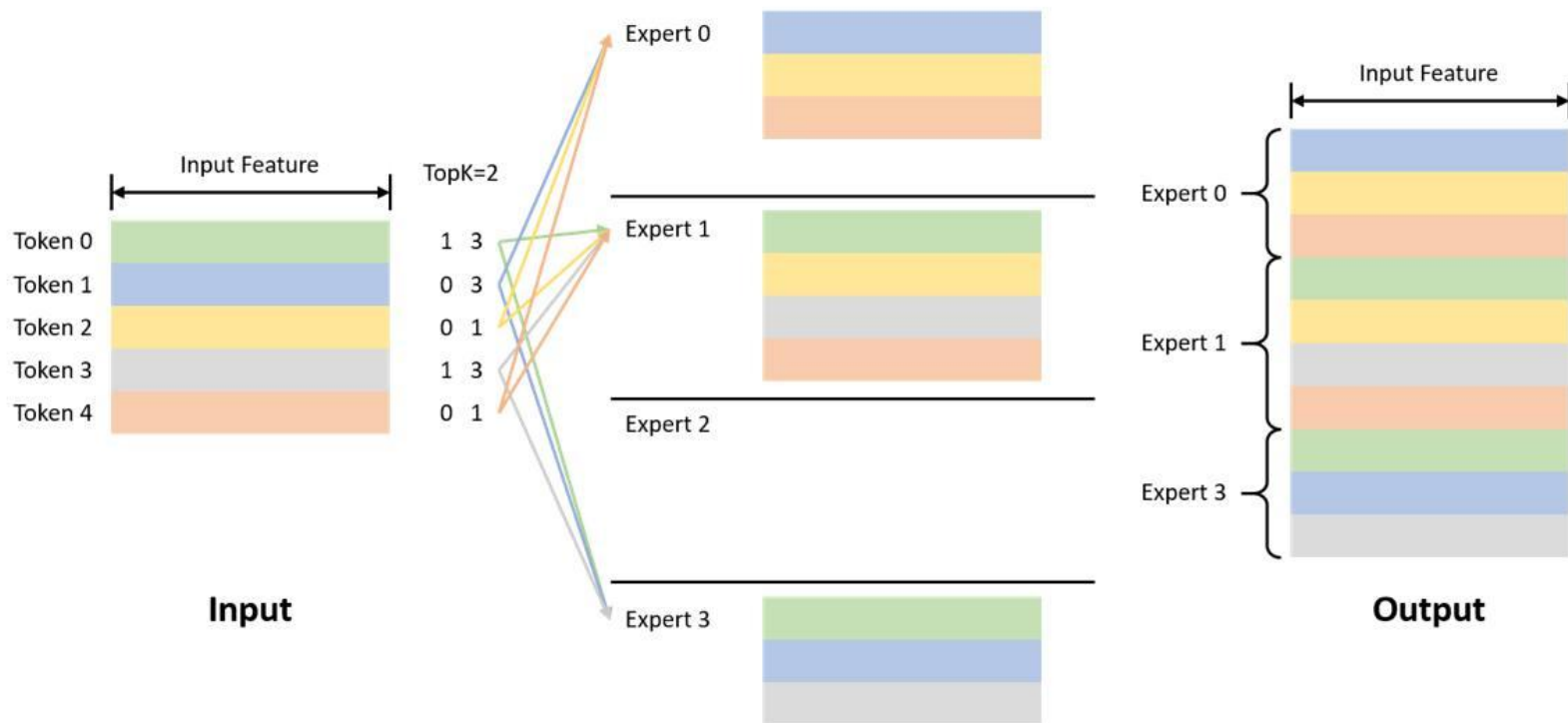


Dense LMs (OLMo, Llama...)



OLMoE





Grouped GEMM

```
gmm

1 def gmm(a, b, batch_sizes):
2     """Grouped GEMM.
3
4     a: (topK * seqlen, hidden)
5     b: (num_experts, hidden, intermediate)."""
6     batch_sizes = batch_sizes().cpu().numpy()
7
8     out = []
9     start = 0
10    for i, size in enumerate(batch_sizes):
11        rhs = b[i, :, :] # i-th expert (hidden, intermediate)
12        result = a[start:start+size] @ b # take `size` consecutive tokens
13        out.append(result) # (size, intermediate)
14        start += size
15
16    return torch.cat(out) # (topK * seqlen, intermediate)
17
```

Grouped GEMM

Токены (seqlen * topk, hidden_size):

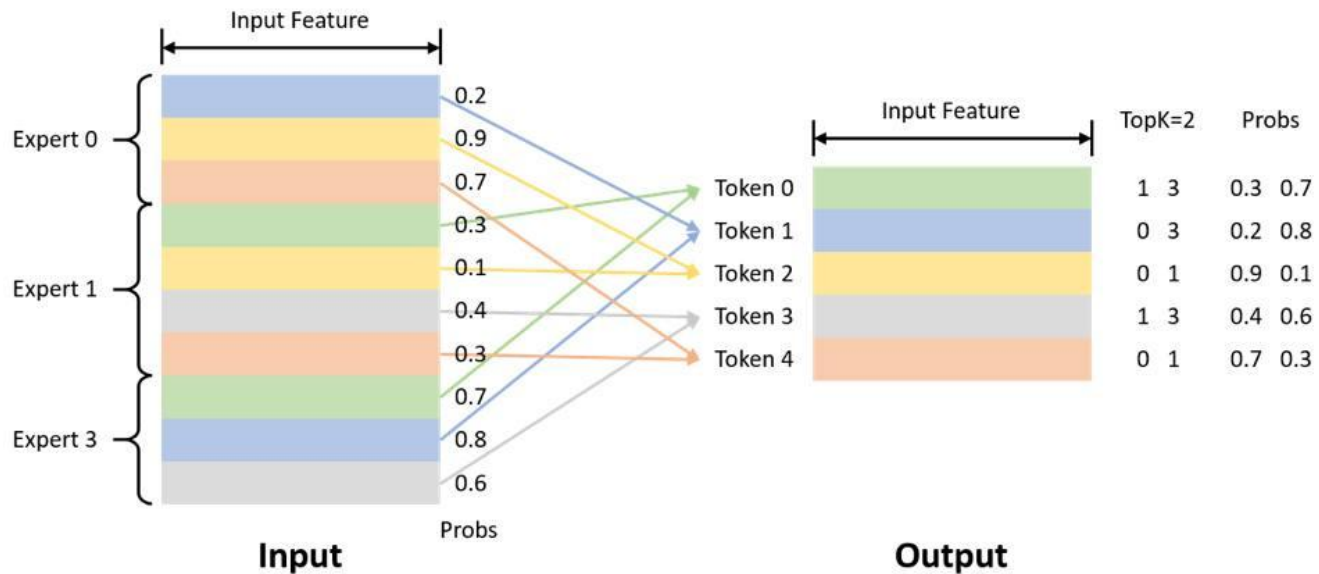
- Сначала токены в 0 эксперта
- Потом в 1ого
- ...

Эксперты:

- (num_experts, hidden_size, inter)
- (num_experts, hidden_size, inter)
- (num_experts, inter, hidden_size)

```
gmm

1 def gmm(a, b, batch_sizes):
2     """Grouped GEMM.
3
4     a: (topK * seqlen, hidden)
5     b: (num_experts, hidden, intermediate)."""
6     batch_sizes = batch_sizes().cpu().numpy()
7
8     out = []
9     start = 0
10    for i, size in enumerate(batch_sizes):
11        rhs = b[i, :, :] # i-th expert (hidden, intermediate)
12        result = a[start:start+size] @ b # take `size` consecutive tokens
13        out.append(result) # (size, intermediate)
14        start += size
15
16    return torch.cat(out) # (topK * seqlen, intermediate)
17
```

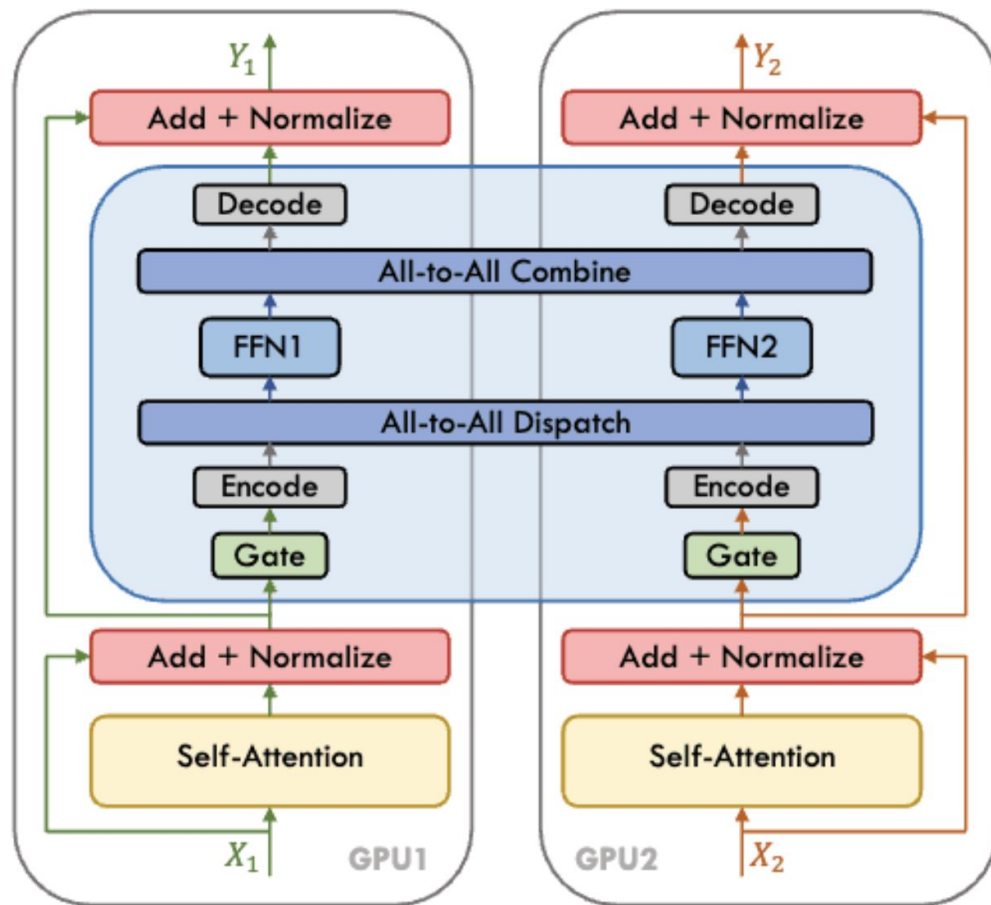


Unpermute Op

Expert Parallel

- Каждый эксперт – независимый dense MLP блок
- Экспертов очень много (64-512)
- В TP мы бы хранили кусочек каждого эксперта на каждом TP-ранке
- В EP мы храним целых экспертов, но только их часть (режем по *num_experts* размерности)

⇒ Надо “роутить” токены между девайсами внутри EP-группы



(a) Data + Expert Parallelism

Какой примитив коммуникации?

All-to-All

- + Меньше памяти
- Динамические шейпы

All-Gather

- + Одинаковый шейп
- Собираем все токены

Зачем ЕР?

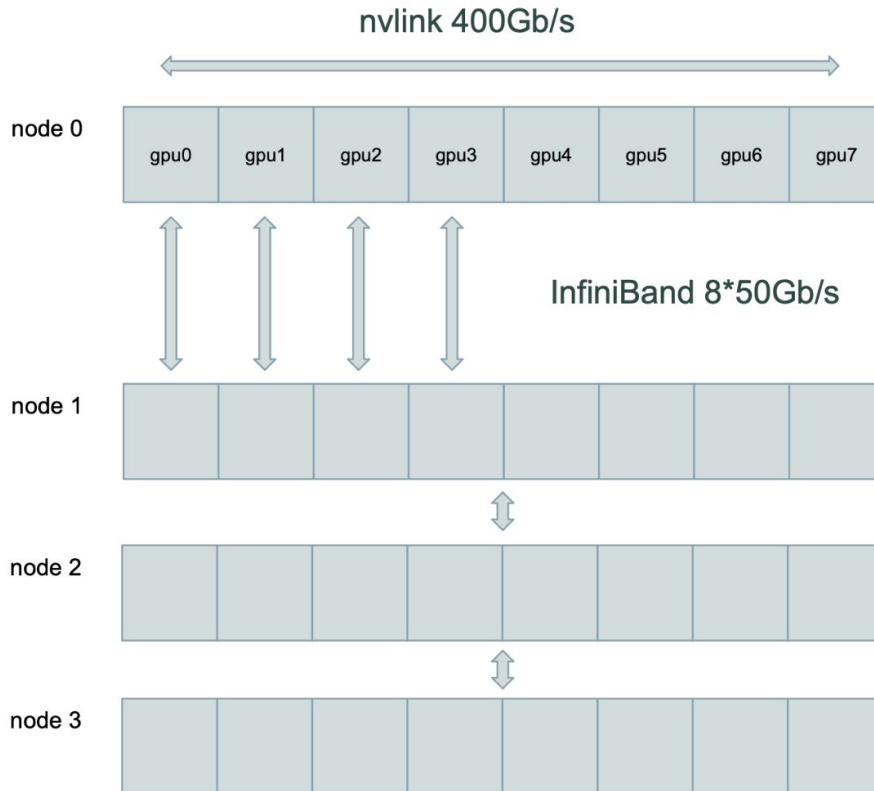
Communications

Setup deepseek-v3

hidden_size = 7168
number_experts_per_tok = 8
n_routed_experts = 256
moe_intermediate_size = 2048

$\sum \text{active} = 8 * 3 * 7168 * 2048 = 352\text{M}$
 $\sum \text{total} = 256 * 3 * 7168 * 2048 = 11.2\text{B}$

(для простоты выкидываем из рассуждений
attention, shared_expert и роутер)



Communications

Setup deepseek-v3

Σ active = 352M

Σ total = 11.2B

seqlen = 8192

h100_flops = 800 TFLOPs bfloat16

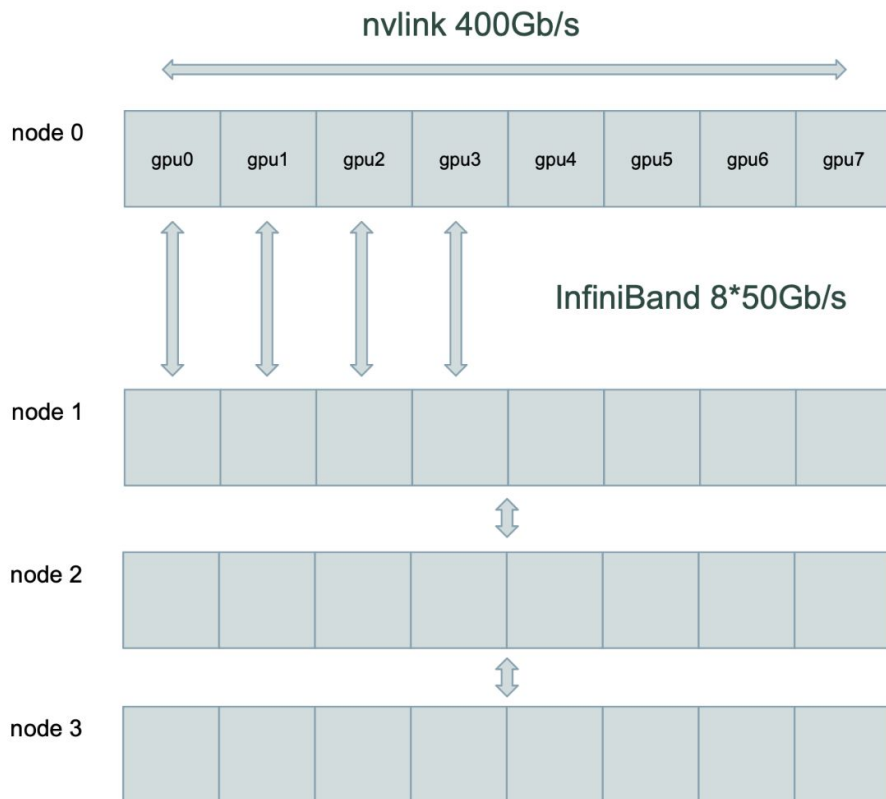
h100_mem = 2.4Tb/s

Вычисления:

1. activation = mbs * seqlen * hidden_size
2. forward = number_experts_per_tok * (seqlen * hidden_size * moe_intermediate_size) / h100_flops =
 $8 * 3 * (8192 * 7162 * 2048) * 2 / 800e12 =$
7ms

Коммуникации:

1. FSDP: 22.4Gb / 400 Gb/s = **56ms**



Communications

Setup deepseek-v3

$\sum \text{active} = 352\text{M}$, $\sum \text{total} = 11.2\text{B}$

$\text{h100_flops} = 800 \text{ TFLOPs}$, $\text{h100_mem} = 2.4\text{Tb/s}$

Tensor Parallel = 8

$\text{batch_size} = 8192 * 8_\text{mbs}$

Вычисления:

1) $7\text{ms} * 8_ \text{mbs} / 8_ \text{tp} = 7\text{ms}$

2) Загрузка активаций:

$8_ \text{mbs} * 8_ \text{topk} * 8192 * 7168 * 2 / 2.4\text{Tb/s} = 3.1\text{ms}$

3) Загрузка весов: $256 * 7168 * 2048 * 2 / 8_ \text{tp} / 2.4\text{Tb/s} = 0.4\text{ms}$

$\sum = 10.5\text{ms}$

Коммуникации:

1. FSDP: $22.4\text{Gb} / 400\text{Gb/s} = 56\text{ms}$

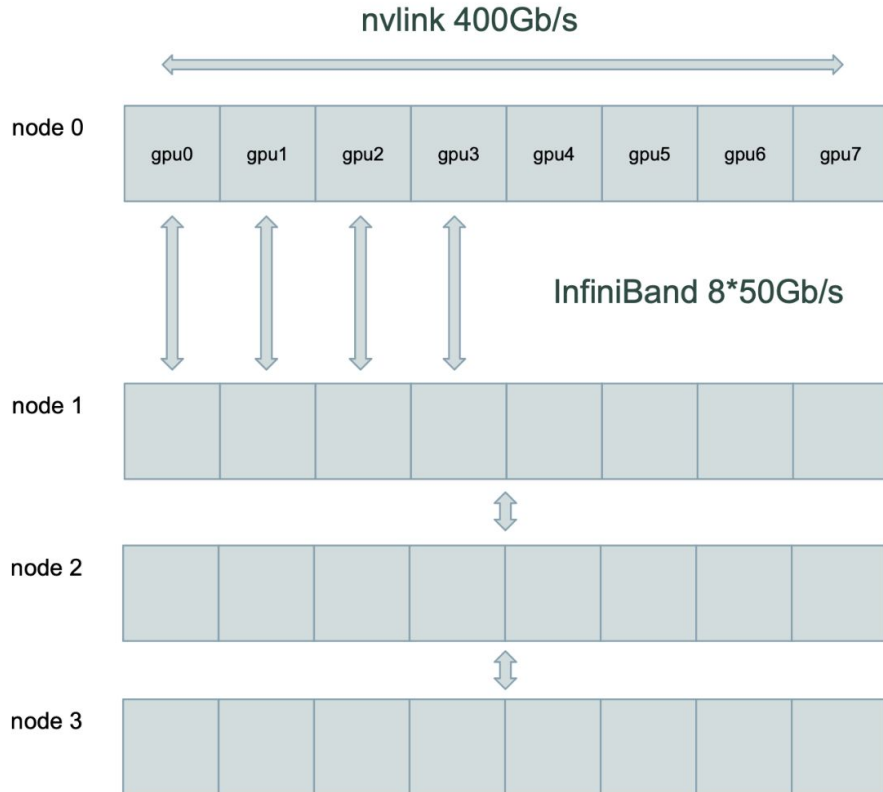
2. AllGather + ReduceScatter = $8_ \text{mbs} * 8192 * 7168 * 2 / 400\text{Gb/s} = 2\text{ms}$

$\sum = 58\text{ms}$

Проблемы:

1) Memory bound при больших TP и mbs

2) FSDP bottleneck при маленьких TP



Communications

Setup deepseek-v3

$\sum \text{active} = 352\text{M}$, $\sum \text{total} = 11.2\text{B}$

$\text{h100_flops} = 800 \text{ TFLOPs b16}$, $\text{h100_mem} = 2.4\text{Tb/s}$

Tensor Parallel Internode= 4

$\text{batch_size} = 8192 * 4_mbs$

Вычисления:

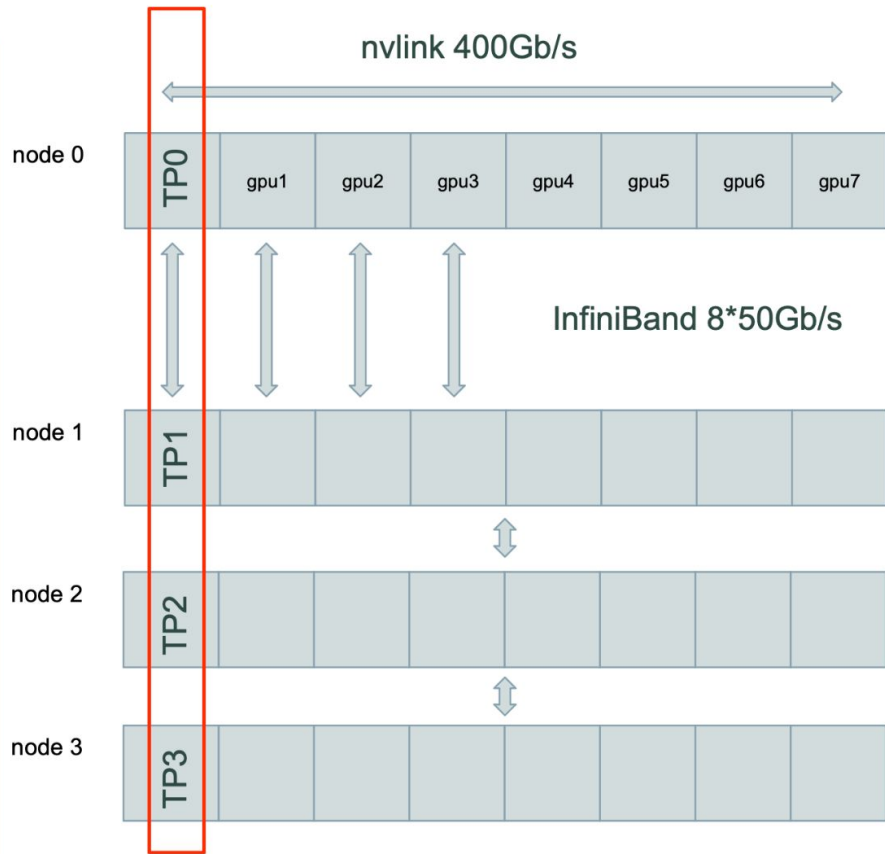
- 1) $7\text{ms} * 4_mbs / 4_tp = 7\text{ms}$
- 2) Загрузка активаций: $4_mbs * 8_topk * 8192 * 7168 * 2 / 2.4\text{Tb/s} = 1.6\text{ms}$
- 3) Загрузка весов: $256 * 7168 * 2048 * 2 / 4_tp / 2.4\text{Tb/s} = 0.8\text{ms}$

$\sum = 9.4\text{ms}$

Коммуникации:

1. FSDP: $22.4\text{Gb} * (\frac{1}{4}) / 400 \text{ Gb/s} = 14\text{ms}$
2. AllGather + ReduceScatter =
 $4_mbs * 8192 * 7168 * 2 * 2_ops / 50\text{Gb/s} = 18\text{ms}$

$\sum = 32\text{ms}$



Communications

Setup deepseek-v3

$\sum$ active = 352M, $\sum$ total = 11.2B

h100_flops = 800 TFLOPs bf16, h100_mem = 2.4Tb/s

Expert Parallel = 8

batch_size = 8192

Вычисления:

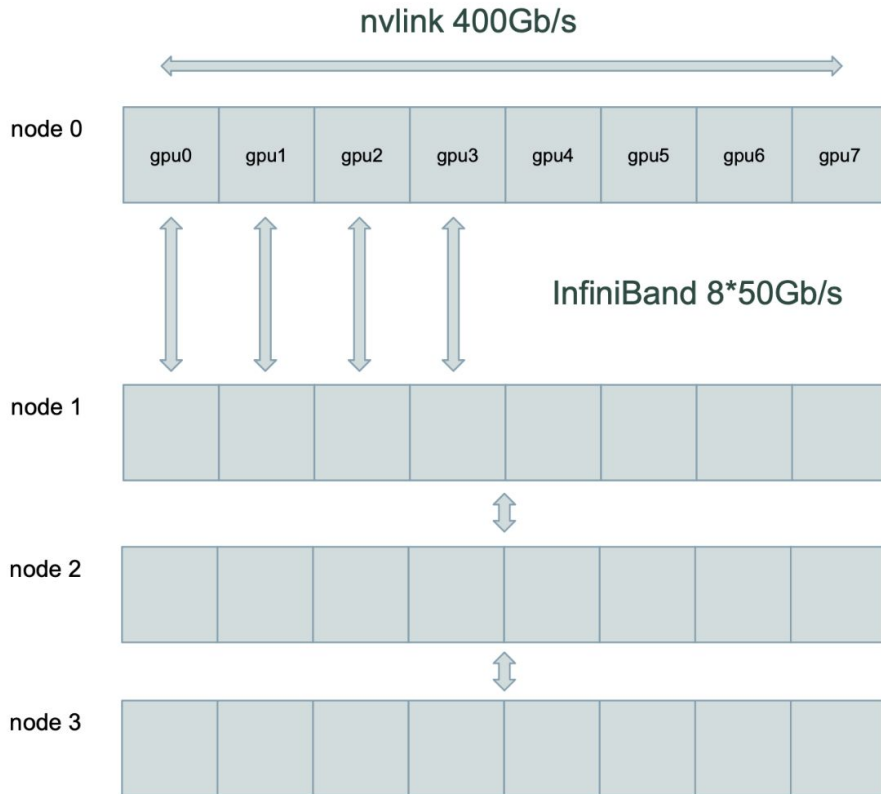
- 1) $8 * 3 * (8192 * 7162 * 2048) * 2 / 800e12 = 7ms$
- 2) Загрузка активаций: $8 * 8192 * 7168 * 2 / 2.4Tb/s = 0.4ms$
- 3) Загрузка весов: $256 * 7168 * 2048 * 2 / 8_ep / 2.4Tb/s = 0.4ms$

$\sum = 7.8ms$

Коммуникации:

1. FSDP: $22.4Gb / 400 Gb/s = 56ms$
2. Dispatch AllGather = $8_ep * 8192 * 7162 * 2 / 400Gb/s = 1.6ms$
3. Combine ReduceScatter = 1.6ms

$\sum = 59.2ms$



Communications

Setup deepseek-v3

$\sum$ active = 352M, $\sum$ total = 11.2B

h100_flops = 800 TFLOPs bf16, h100_mem = 2.4Tb/s

Expert Parallel = 16

batch_size = 8192

Вычисления:

- 1) 7ms
- 2) Загрузка активаций: 0.4ms
- 3) Загрузка весов: $256 * 7168 * 2048 * 2 / 16_ep / 2.4Tb/s = 0.2ms$

$\sum = 7.6ms$

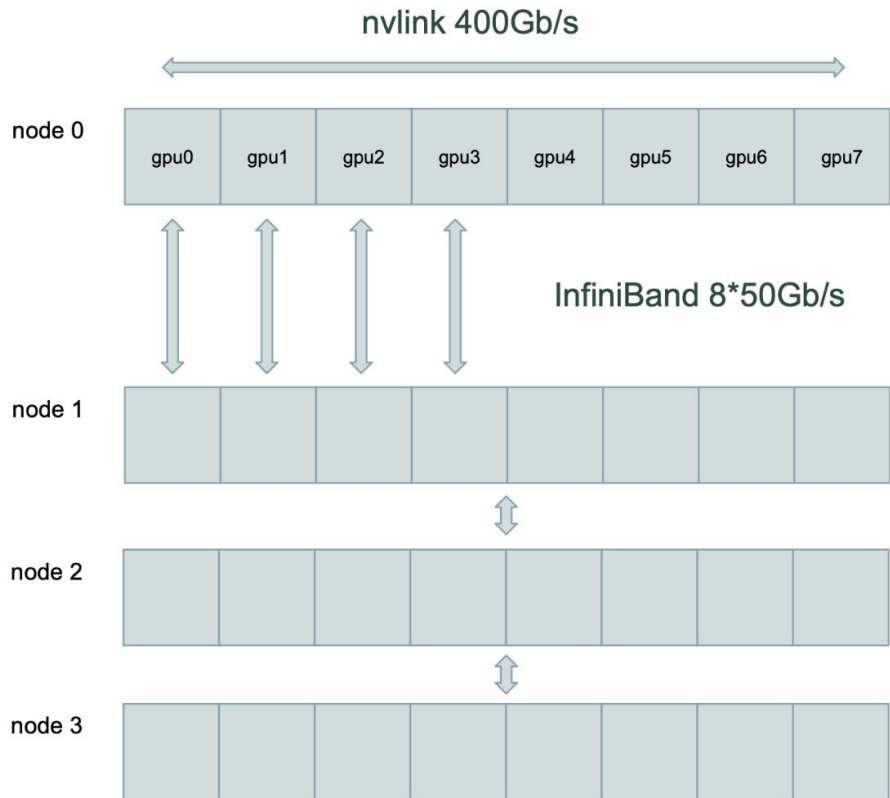
Коммуникации:

1. $22.4Gb * (\frac{1}{2}) / 400 Gb/s = 28ms$
2. Dispatch - AllGather = $16_ep * 8192 * 7162 * 2 / 400Gb/s = 3.2ms$
3. Combine - ReduceScatter = 3.2ms

$\sum = 34.4ms$

Выводы:

1. Лучше скейлится
2. Экономит память - об этом дальше

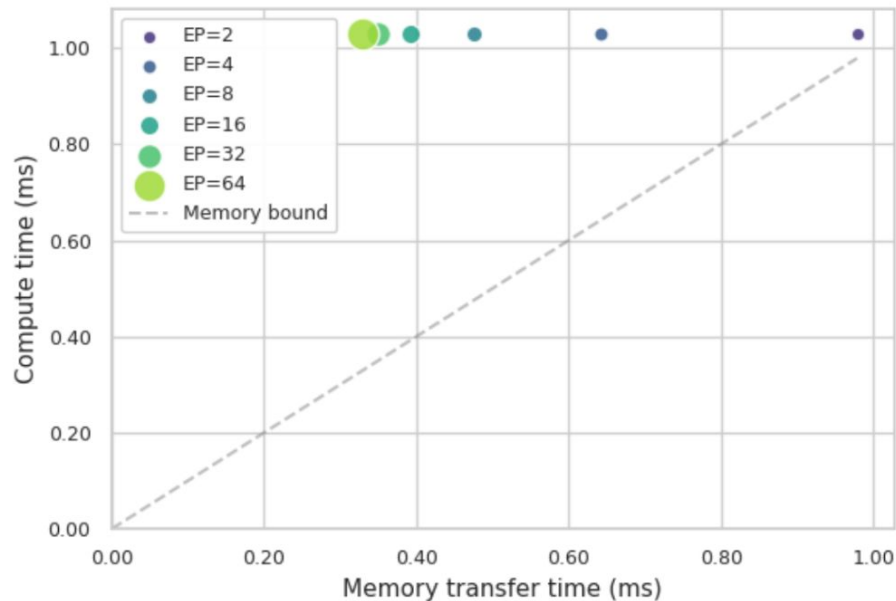


Входы: $(\text{num_experts}/EP, \text{mbs} \cdot \text{seqlen} \cdot (\text{top_k} / \text{num_experts}) * EP, \text{hidden})$

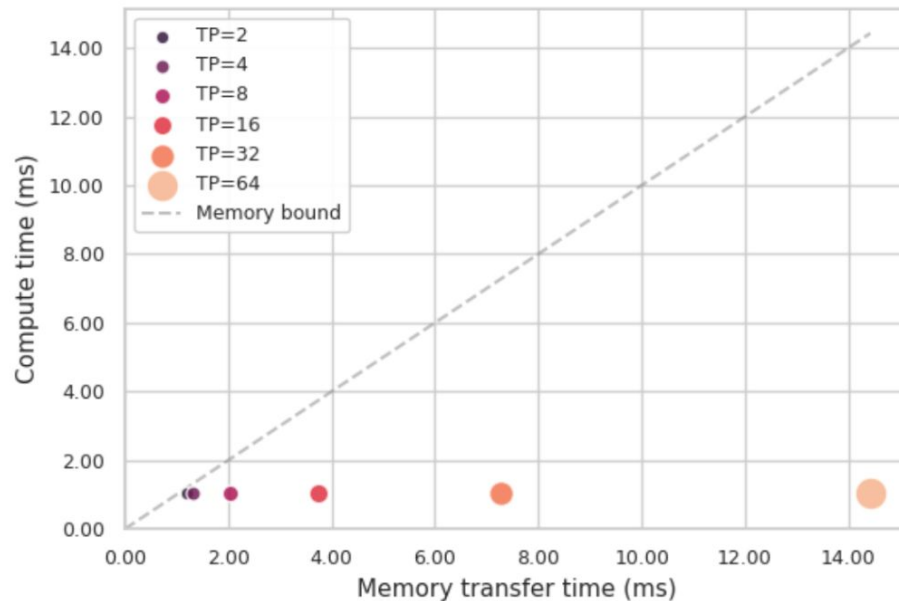
Веса: $(\text{num_experts} / EP * \text{intermediate} / TP, \text{hidden})$

$\text{mbs}=TP$, чтобы не резать батч

Expert Parallelism (EP)



Tensor Parallelism (TP)

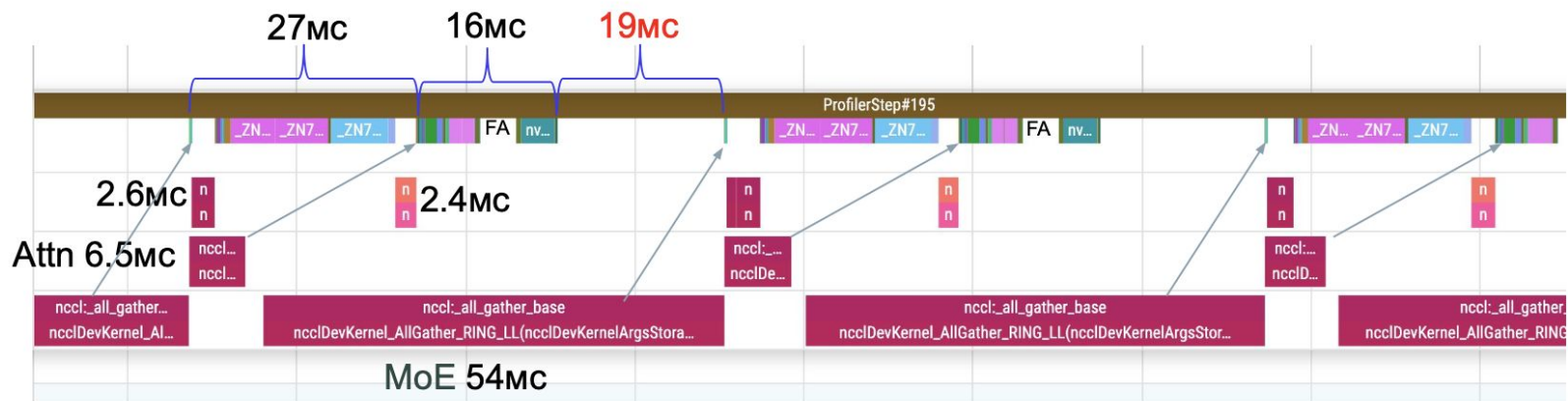


Входы: $(\text{num_experts}/\text{EP}, \text{mbs} \cdot \text{seq_len} \cdot (\text{top_k} / \text{num_experts}) \cdot \text{EP}, \text{hidden})$

Беса: $(\text{num_experts} / \text{EP} \cdot \text{intermediate} / \text{TP}, \text{hidden})$

mbs=TP, чтобы не резать батч

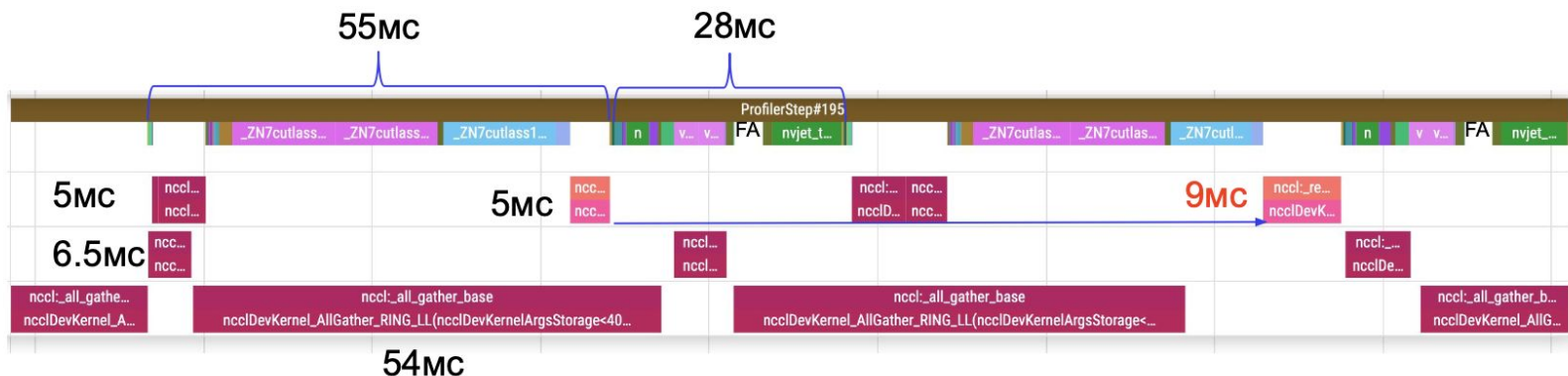
EP на практике



Setup deepseek-v3

- seqlen 8192
- fsdp zero3
- Full checkpointing
- bfloat16
- mbs1 ep1 = OOM
- mbs1 ep2 = OOM
- mbs1 ep8 = 62ms

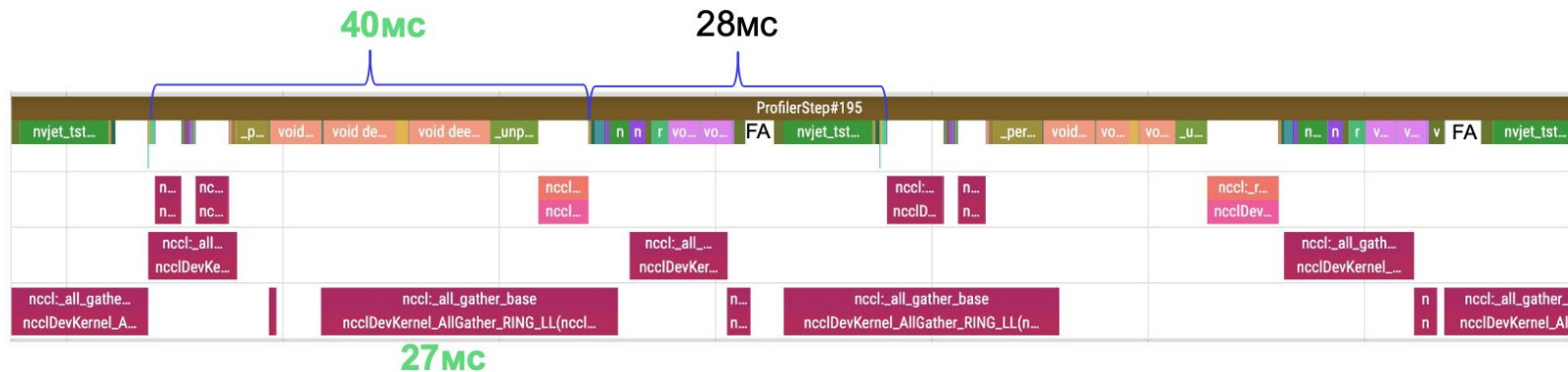
ЕР на практике



Setup deepseek-v3

- seqn 8192
- fsdp zero3
- Full checkpointing
- bfloat16
- mbs1 ep1 = OOM
- mbs1 ep2 = OOM
- mbs1 ep8 = 62ms
- Mbs2 ep8 = 83ms (42ms / mbs1)

EP на практике



Setup deepseek-v3

- seqlen 8192
- fsdp zero3
- Full checkpointing
- bfloat16
- fp8 MoE and FSDP
- mbs1 ep1 = OOM
- mbs1 ep2 = OOM
- mbs1 ep8 = 62ms
- Mbs2 ep8 = 83ms
- Mbs2 ep8 fp8 = 68ms (34ms / mbs1)

Context Parallel

Long context



[Send feedback](#)

Many Gemini models come with large context windows of 1 million or more tokens. Historically, large language models (LLMs) were significantly limited by the amount of text (or tokens) that could be passed to the model at one time. The Gemini long context window unlocks many new use cases and developer paradigms.

Grok 4 Fast

We're excited to release grok-4-fast, our latest advancement in cost-efficient reasoning models.

Modalities	Context window
T → T	2,000,000

Features

- Function calling
- Structured outputs
- Reasoning
- Lightning fast
- Low cost

[View base model](#)[View non-reasoning model](#)



1M token context window

Claude Sonnet 4 and 4.5 support a 1-million token context window. This extended context window allows you to process much larger documents, maintain longer conversations, and work with more extensive codebases.

The 1M token context window is currently in beta for organizations in [usage tier 4](#) and organizations with custom rate limits. The 1M token context window is only available for Claude Sonnet 4 and Sonnet 4.5.

Зачем?

CoT в ризонинге

Агенты

Код

CoT в ризонинге

Агенты

Код

Активации больш

Распилим вход на независимые кусочки?

Transformer

MLP = справочник, независимо обогатим токен информацией

Transformer

MLP = справочник, независимо обогатим токен информацией

Attention = учтем контекстную семантику

Transformer

MLP = справочник, независимо обогатим токен информацией

Attention = учтем контекстную семантику

Softmax attention:

Parallel training : $\mathbf{O} = \text{softmax}(\mathbf{QK}^\top \odot \mathbf{M})\mathbf{V} \in \mathbb{R}^{L \times d}$

Iterative inference : $\mathbf{o}_t = \sum_{j=1}^t \frac{\exp(\mathbf{q}_t^\top \mathbf{k}_j)}{\sum_{l=1}^t \exp(\mathbf{q}_t^\top \mathbf{k}_l)} \mathbf{v}_j \in \mathbb{R}^d$

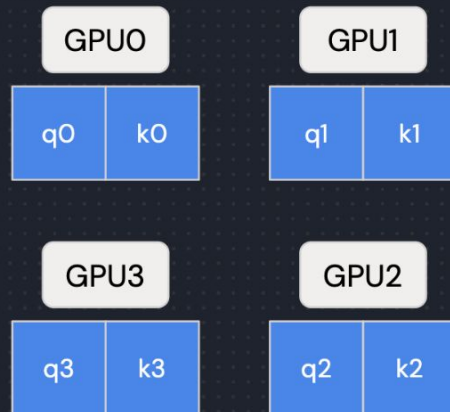
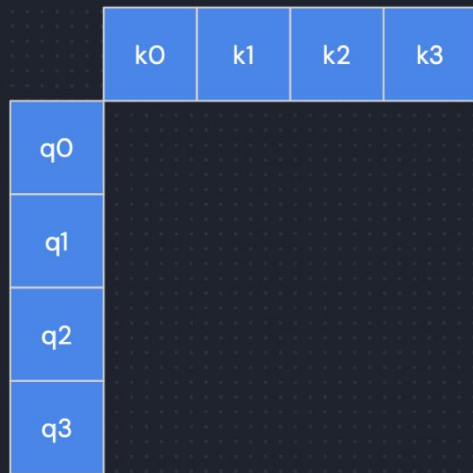
where $\mathbf{M} \in \mathbb{R}^{L \times L}$ is the casual mask:

$$\mathbf{M}_{i,j} = \begin{cases} -\infty & \text{if } j > i \\ 1 & \text{if } j \leq i \end{cases}$$

CP = режим по SeqLen-у, но добавляем
коммуникацию в аттеншн

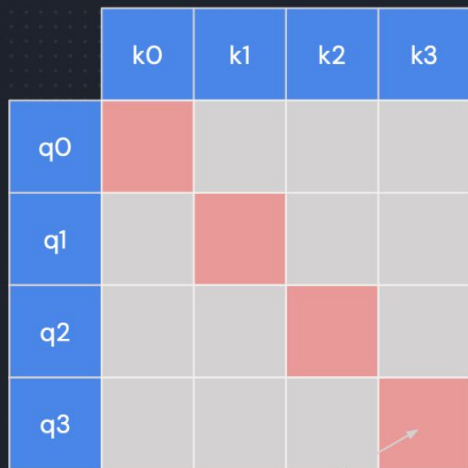
Ring attention

num_chunks=4
(CP=4)

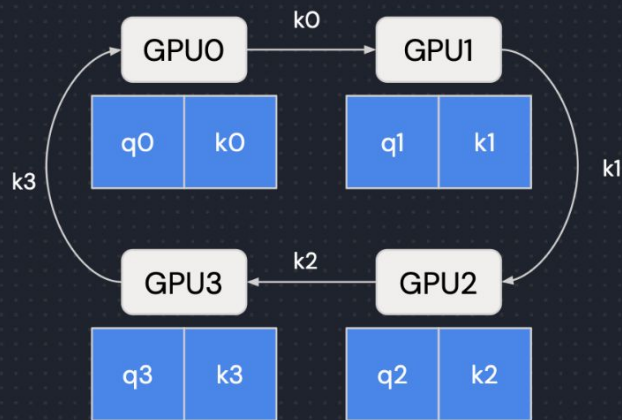


Ring attention

num_chunks=4
(CP=4)

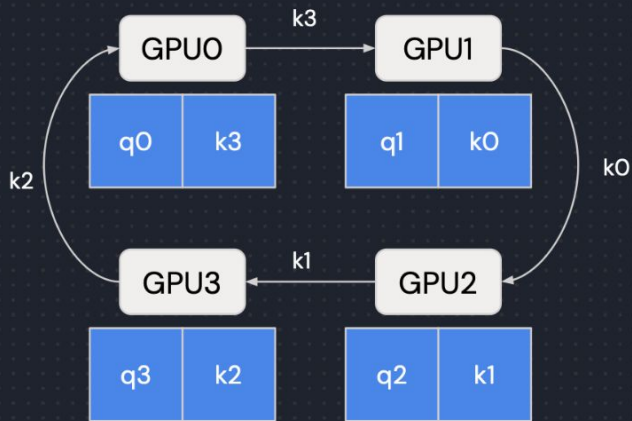
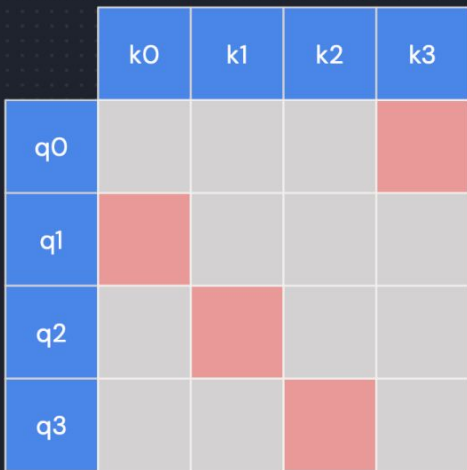


FlashAttention



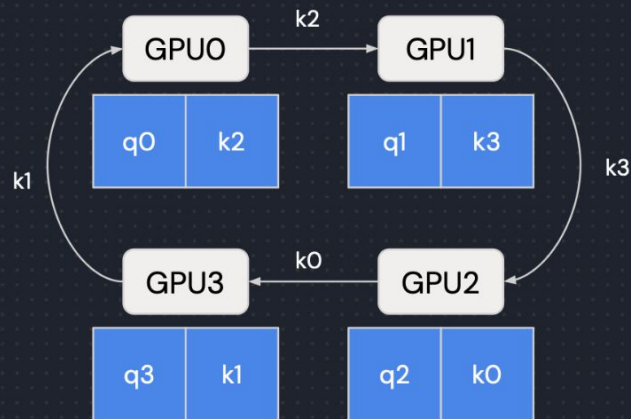
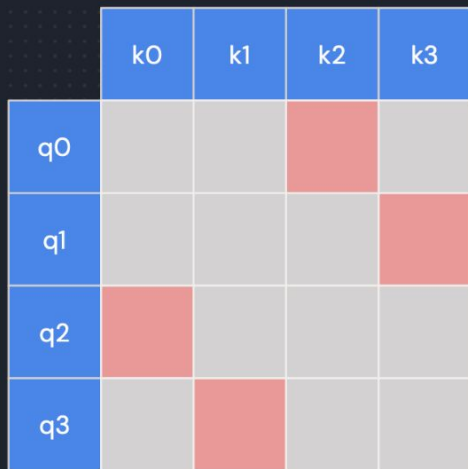
Step #0

Ring attention



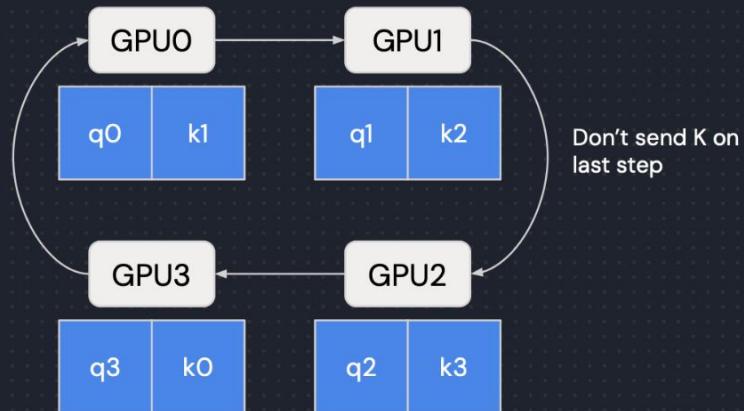
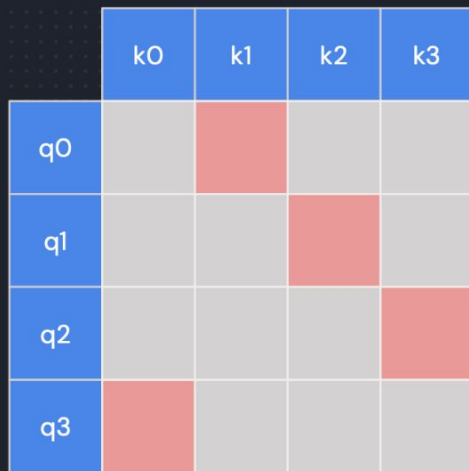
Step #1

Ring attention



Step #2

Ring attention



Step #3

Ring attention

	k0	k1	k2	k3
q0	out, lse	out, lse	out, lse	out, lse
q1	out, lse	—		
q2				
q3				

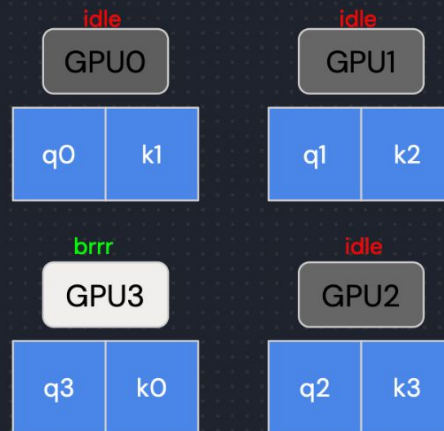
Each step produced **out** and **lse**

We should use **Online Softmax**

online softmax

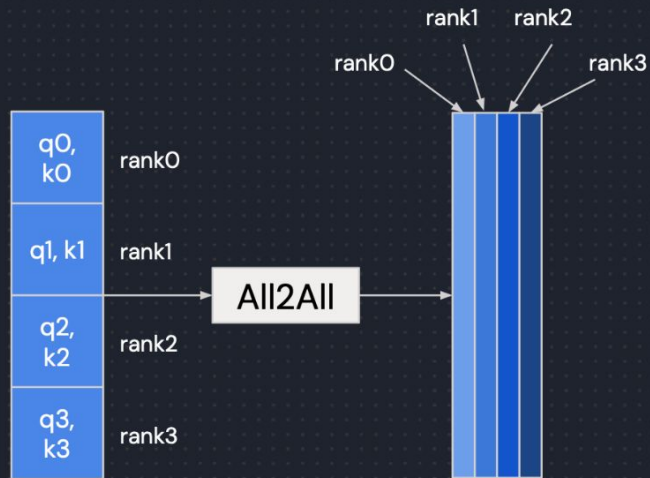
Aggregation

Ring attention



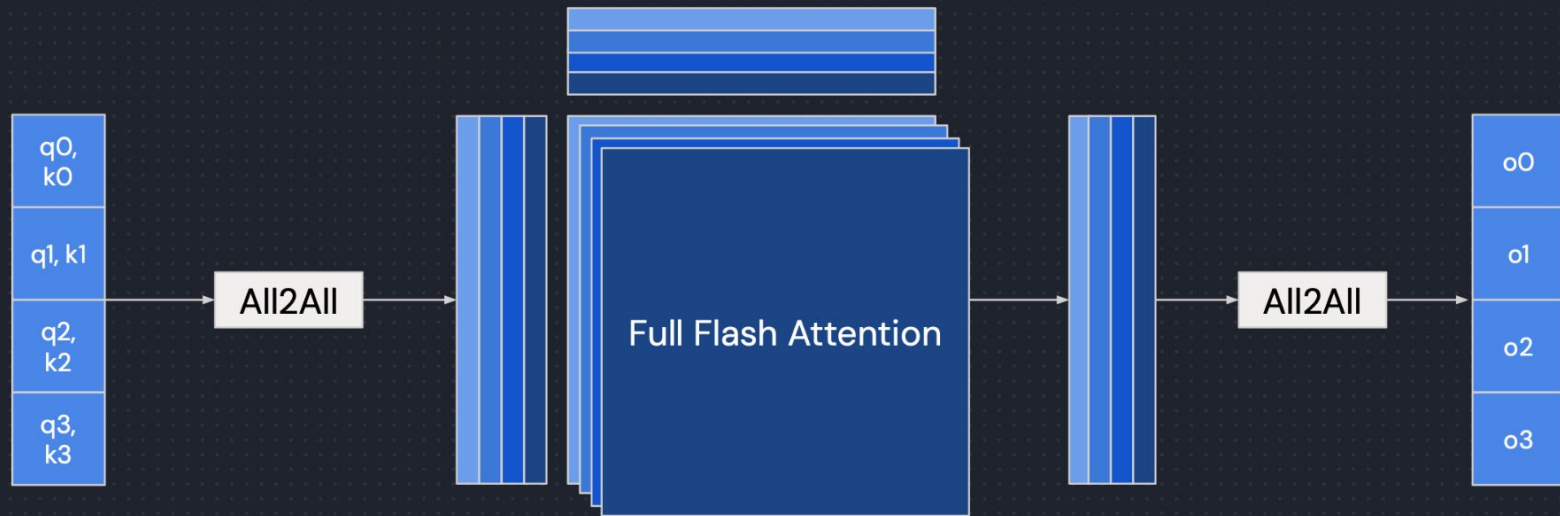
Step #3

DeepSpeed Ulysses



What if we divide by num_heads dim?

DeepSpeed Ulysses



What if we divide by num_heads dim?

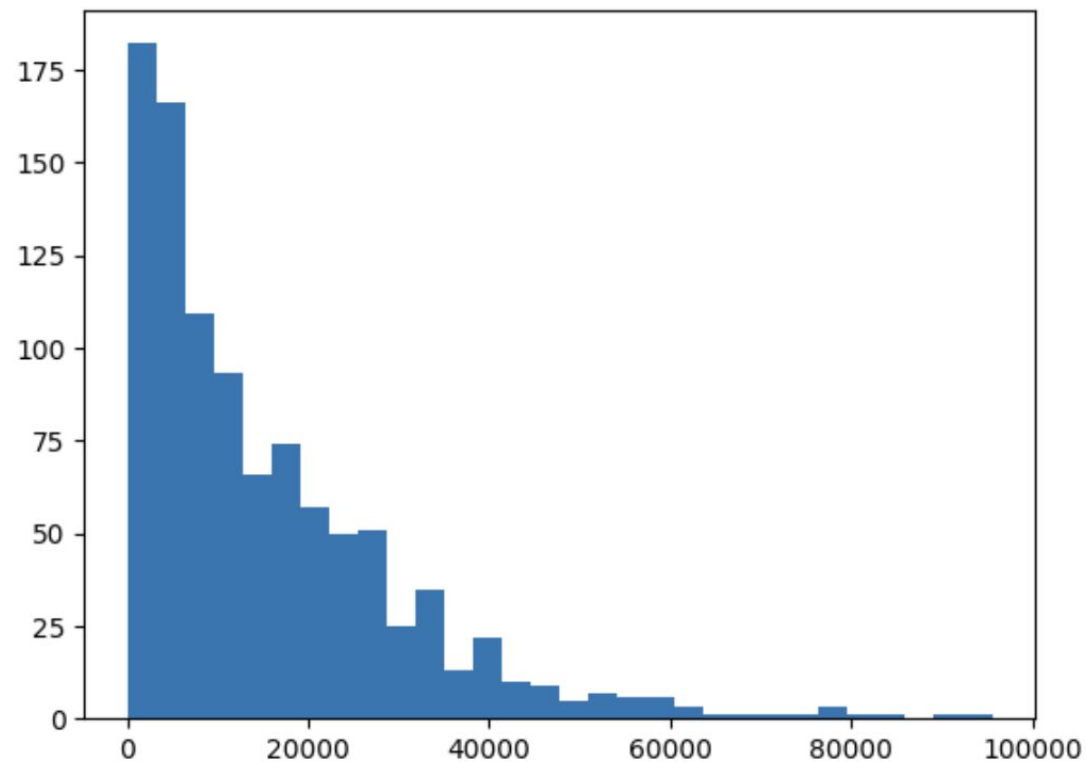
Ulysses

2 All-to-All на слой

Дорого между хостами

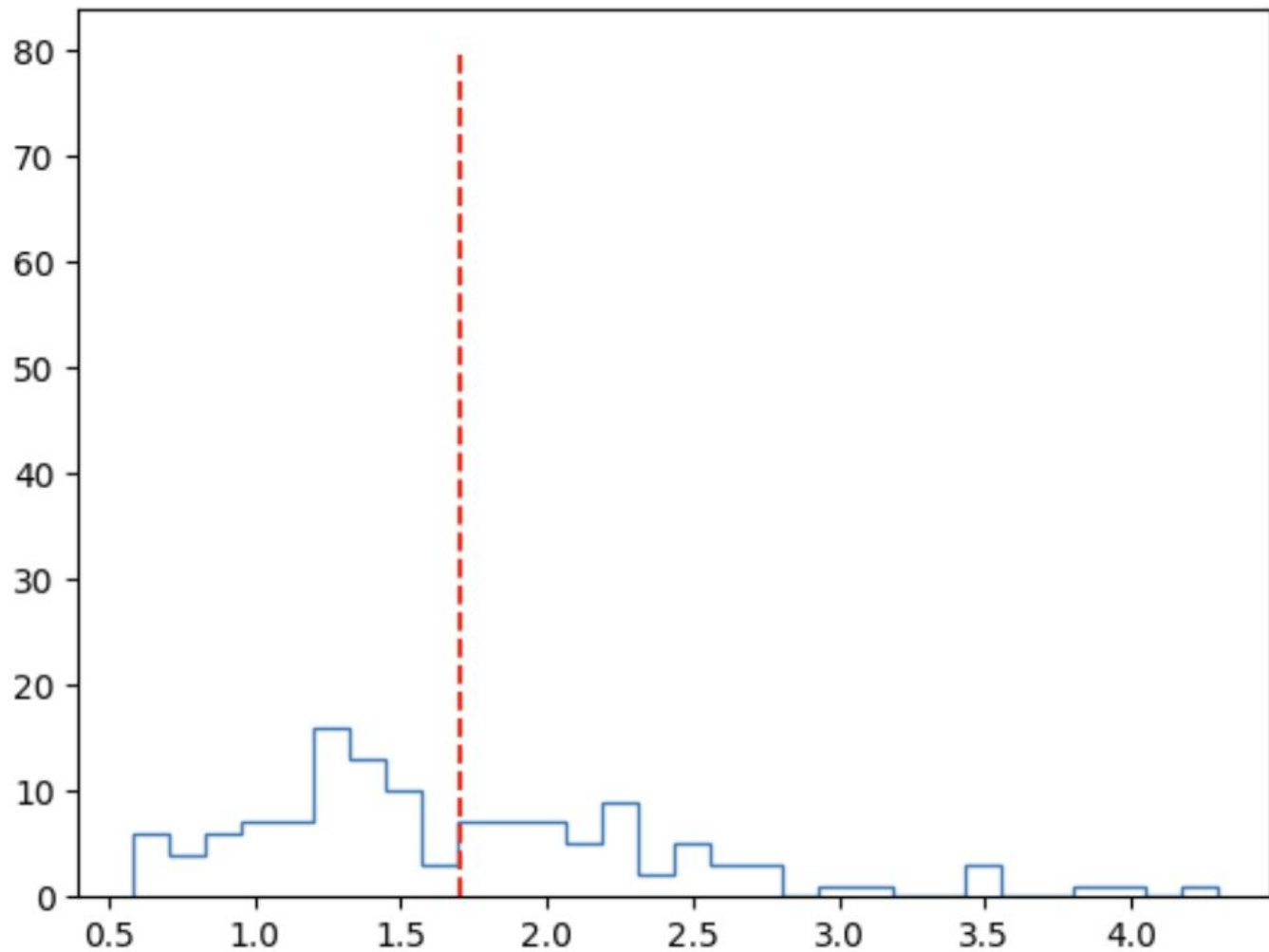
CP выравнивает нагрузку

Распределение данных

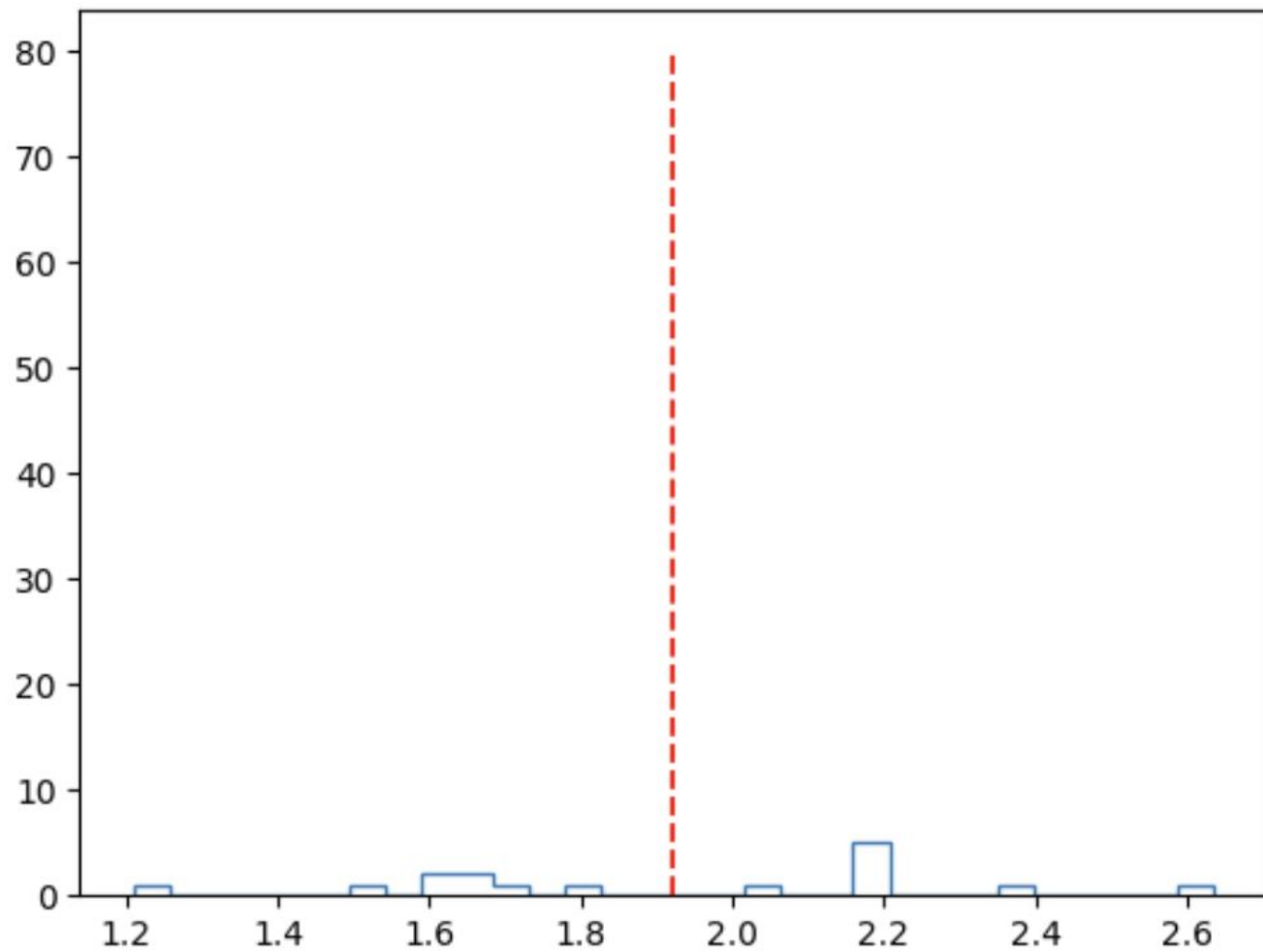


```
1  for cp in (1, 8, 16):
2
3      batches = []
4
5      max_seq_len = 65536
6      cp_seq_len = cp * max_seq_len
7      DP_noCP = 128 // cp
8
9      for i in range(100):
10         global_batch = []
11         for p in range(DP_noCP):
12             sum_ = 0
13             batch = []
14             while sum_ < cp_seq_len:
15                 new_sample = int(min(np.random.exponential(max_seq_len/4), max_seq_len))
16                 sum_ += new_sample
17                 if sum_ > cp_seq_len:
18                     mod = sum_%cp_seq_len
19                     new_sample -= mod
20                 batch.append(new_sample)
21             global_batch.append(batch)
22         batches.append(global_batch)
23
24     for i in range(len(batches)):
25         for j in range(len(batches[i])):
26             batches[i][j] = sum(map(lambda x: x*x, batches[i][j])) / cp
27
28     i = 0
29     plt.hist(batches[i], bins=30, histtype='step');
30     mean = sum(batches[i])/len(batches[i])
31     plt.plot([mean, mean], [0, 80], 'r--')
32     plt.show()
```

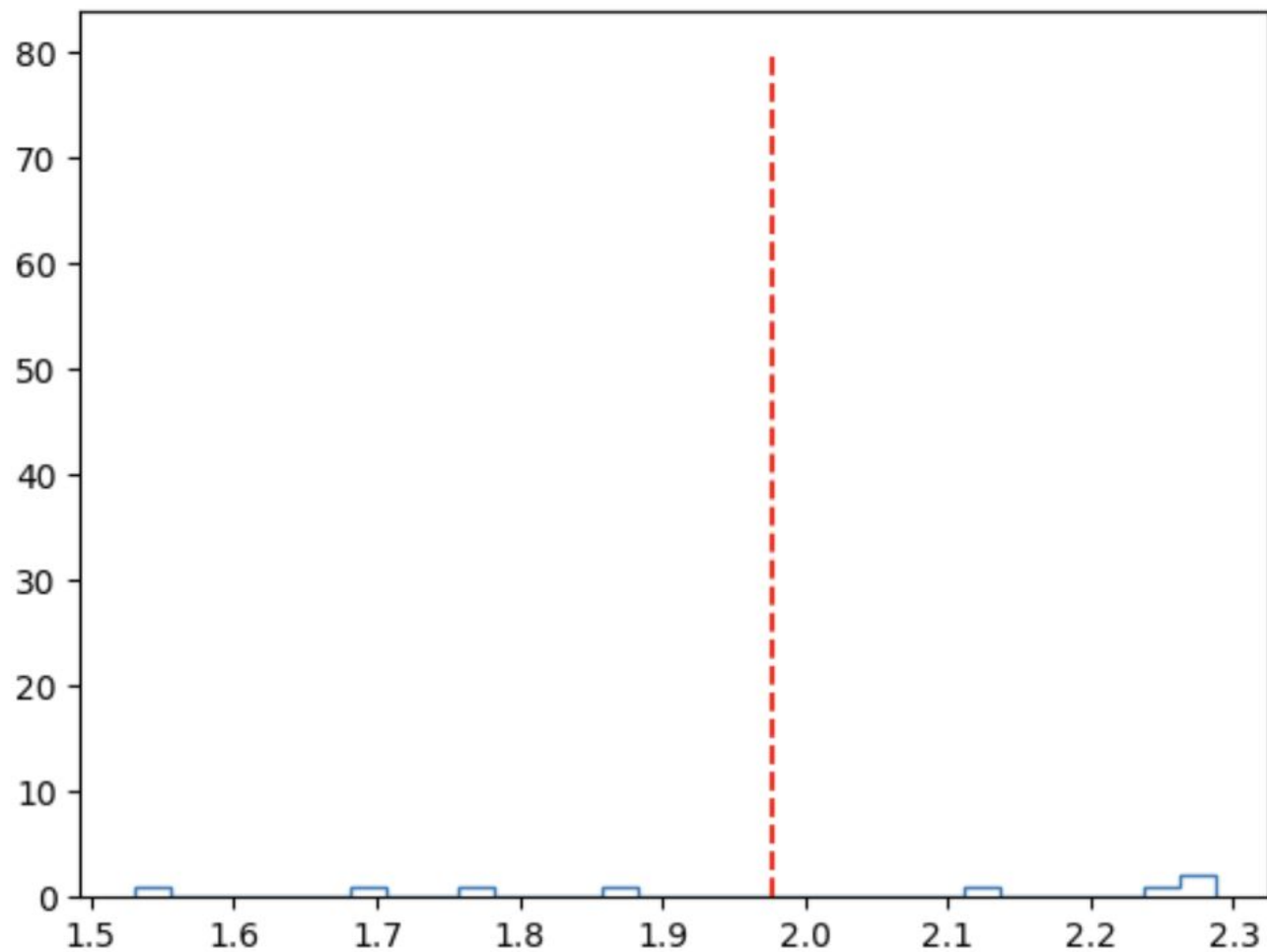
CP1



CP8



CP16



- FSDP не всегда хватает:
 - GPU-poor: Активации могут не влезать
 - GPU-rich: Можем пробить Critical Batch Size и испортить качество
 - GPU-rich: Если не пробили CBS, то будем communication-bound, не утилизируем железо
- Tensor Parallel:
 - Column Parallel vs. Row Parallel – умножаем шардированные матрицы
 - Чистый TP вставляем в Attention и MLP, появляются два All Reduce
 - Дропауту и LN все еще нужен полный хидден
 - **Внутри одной ноды, иначе огромный оверхед на коммуникации**
- Tensor Parallel + Sequence Parallel:
 - Режим входы в не-матмул операции по секлену
 - All Reduce разменивается на All Gather + Reduce Scatter
- Expert Parallel:
 - Для MoE
 - Эксперты отселяются отдельно, добавляется раскидывание токенов между экспертами. Можно межхостовый. Это дешевле, чем в TP.
- Context Parallel:
 - Режим вход по секлену, никогда не собираем целиком
 - Добавляем коммуникации в аттеншн
 - Помогает бороться с дисбалансом в вычислениях attention